



**DEZVOLTAREA UNUI ASISTENT PERSONAL CU
CAPABILITĂȚI DE CONVERSAȚIE UMANĂ ȘI
RĂSPUNSURI BAZATE PE EMOȚII**

LUCRARE DE LICENȚĂ

Absolvent: **Paul-Teodor CRIȘAN**

Coordonator
științific: **Conf. Dr. Ing. Decan Camelia LEMNARU**

2024



FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DECAN,
Prof. dr. ing. Liviu MICLEA

DIRECTOR DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Absolvent: **Paul-Teodor CRIȘAN**

**DEZVOLTAREA UNUI ASISTENT PERSONAL CU CAPABILITĂȚI DE
CONVERSAȚIE UMANĂ ȘI RĂSPUNSURI BAZATE PE EMOȚII**

1. **Enunțul temei:** Proiectul propus vizează dezvoltarea unui asistent personal, cât mai apropiat de comportamentul uman, capabil să automatizeze sarcini de zi cu zi, integrând diverse funcționalități uzuale. Asistentul ar trebui să fie capabil de dialog general cu utilizatorul, furnizând răspunsuri la întrebări și comenzi în funcție de starea sa de spirit, influențată de emoțiile și intențiile din spatele limbajului utilizatorului.
2. **Conținutul lucrării:** *Introducere, Obiectivele Proiectului, Studiu Bibliografic, Analiză și Fundamentare Teoretică, Proiectare de Detaliu și Implementare, Testare și Validare, Manual de Instalare și Utilizare, Concluzii, Bibliografie*
3. **Locul documentării:** Universitatea Tehnică din Cluj-Napoca, Departamentul Calculatoare
4. **Consultanți:**
5. **Data emiterii temei:** 1 Noiembrie 2023
6. **Data predării:** 12 iulie 2024

Absolvent: _____

Coordonator științific: _____

Cuprins

Capitolul 1. Introducere	1
1.1. Contextul Proiectului	1
1.2. Prezentarea Generală a Domeniului	2
Capitolul 2. Obiectivele Proiectului	3
2.1. Obiective Principale.....	3
2.2. Obiective Secundare	4
Capitolul 3. Studiu Bibliografic.....	5
3.1. Asistenți Personali Inteligenți și Recunoașterea Vocală	5
3.2. Încrederea Utilizatorilor și Securitatea Asistenților Personali Inteligenți	5
3.3. Integrarea Inteligenței Artificiale și Învățării Automate	6
3.4. Framework-ul Rasa.....	6
3.4.1. Rasa NLU	6
3.4.2. Rasa Core.....	7
3.4.3. Flexibilitatea și Utilitatea Rasa	7
3.5. Transformers.....	7
3.6. Modele de Limbaj de Mari Dimensiuni (LLMs).....	8
3.6.1. Fine-tuning.....	8
3.7. Detectarea Intențiilor și Completarea Sloturilor	8
3.8. Sisteme Text-to-Speech (TTS)	9
Capitolul 4. Analiză și Fundamentare Teoretică.....	11
4.1. Arhitectura Conceptuala	11
4.2. Înțelegerea Limbajului Natural	12
4.2.1. Tokenizarea	13
4.2.2. Vectorizarea.....	13
4.2.3. Caracterizarea (Featurizing)	14
4.2.4. Detectia Intențiilor & Extragerea Entităților	15
4.2.5. Completarea Parametrilor Intenției (Slot Filling).....	16
4.2.6. Selecția Răspunsului.....	16
4.3. Modele Mari de Limbaj	16
4.3.1. Arhitectura Transformer	17
4.3.2. Integrarea unui LLM într-un Asistent Personal Inteligent	19
4.3.3. Procesul de Reglare Fină	19
4.4. Metrice de Evaluare.....	20

Capitolul 5. Proiectare de Detaliu și Implementare	22
5.1. Schema Generală a Aplicației.....	22
5.1.1. Fluxul de Procesare a Comenzilor	23
5.2. Controlorul de Mesaje	25
5.3. Nucleul Rasa	26
5.3.1. Tokenizarea	26
5.3.2. Caracterizarea	27
5.3.3. Clasificarea Intențiilor și Extragerea Entităților	27
5.3.4. Maparea Sinonimelor	28
5.3.5. Selectarea Răspunsului	29
5.3.6. Structura Nucleului Rasa	29
5.3.7. Acțiunile Rasa.....	31
5.4. Serviciul de Reamintiri (Reminders)	34
5.4.1. Intenția ‘set_reminder’	34
5.4.2. Serverul de Remindere	35
5.4.3. Intenția ‘reminder_notification’	35
5.4.4. Argumentarea Decuplării Serviciului	35
5.5. Componenta de Umanizare.....	36
5.5.1. Conversația Umana.....	36
5.5.2. Reformularea Bazată pe Emoții.....	38
5.5.3. Conversia Text-Vorbire (Text-to-Speech).....	40
5.6. Comunicarea între Componente	40
5.6.1. Fluxul de comunicare între componente	41
5.6.2. VPN-ul.....	42
Capitolul 6. Testare și Validare	43
6.1. Testarea Nucleului	43
6.1.1. Clasificarea Intențiilor	43
6.1.2. Extragerea Entităților.....	47
6.2. Testarea Algoritmilor Ajutători	49
6.2.1. Convertorul de Timp	50
6.2.2. OpenWeatherMap API	50
6.3. Evaluarea Modelelor de Limbaj	51
6.3.1. Modelul de Dialog	51
6.3.2. Modelul de Reformulare.....	52
6.4. Testarea Experienței Utilizatorului.....	53
6.4.1. Scenariul de Test	53

Capitolul 7. Manual de Instalare si Utilizare	56
7.1. Resursele Hardware	56
7.2. Resursele Software	56
7.2.1. Nucleul.....	56
7.2.2. VPN-ul.....	57
7.2.3. Serverul de Umanizare	57
7.3. Rulare și Utilizare	58
Capitolul 8. Concluzii	59
Bibliografie	61

Capitolul 1. Introducere

1.1. Contextul Proiectului

Inteligența artificială (AI) și învățarea automata (eg. Machine Learning, ML) sunt acum esențiale în avansul tehnologic, schimbând modul în care funcționează multe sectoare ale societății. Aceste tehnologii au revoluționat industrii precum sănătatea și educația, prin automatizare și personalizare, și au îmbunătățit semnificativ producția și serviciile. Creșterea rapidă a AI a dus la inovații care fac procesele mai eficiente și îmbunătățesc calitatea vieții, arătând cât de importantă este AI în interacțiunile noastre zilnice.

Utilizarea AI în dispozitivele de zi cu zi a schimbat relația noastră cu tehnologia. Asistenți personali ca Alexa și Siri, care folosesc recunoaștere vocală și procesare a limbajului natural, sunt exemple clare ale acestei integrări. Recent, modelele de limbaj de mare capacitate, cum ar fi GPT-3, au schimbat modul în care computerele înțeleg și generează limbajul uman, permițând sisteme de conversație mai sofisticate și personalizate. Progresul în tehnologia LLM-urilor deschide noi posibilități pentru crearea de asistenți AI care nu doar răspund la comenzi, ci și anticipează nevoile utilizatorilor în funcție de context.

Pentru a rula modele de limbaj mari (LLM), cum sunt GPT-3, sunt necesare resurse semnificative, de la hardware avansat la servere puternice și multă energie electrică. Aceste modele necesită o putere de procesare și o capacitate de stocare uriașă, având miliarde de parametri. Evoluția hardware-ului a permis extinderea acestor sisteme, dar costurile și complexitatea logistică rămân prea mari pentru utilizatorii obișnuiți. De aceea, accesul la aceste tehnologii este posibil mai ales prin servicii de cloud, care folosesc resurse distribuite și scalabile. În acest context, marile corporații de tehnologie au un avantaj clar, având resursele necesare pentru a gestiona aceste infrastructuri complexe. Oamenii de rând nu au capacitatea hardware necesară pentru a opera local acești giganți AI, ceea ce accentuează diferențele tehnologice între corporații și consumatorii individuali. Acest peisaj scoate în evidență problemele de accesibilitate și echitate în utilizarea tehnologiilor avansate și subliniază nevoia de soluții care să democratizeze accesul la inovațiile AI.

Cursa pentru dezvoltarea AI este dominată de marile corporații, care au o influență majoră asupra direcției în care se dezvoltă această tehnologie. Aceste companii nu doar că controlează inovațiile, dar au și monopolizat piața AI. Produsele lor sunt revoluționare și au îmbunătățit modul în care trăim și lucrăm. Sunt unul dintre mulții oameni care apreciază beneficiile aduse de instrumente ca ChatGPT, care ne ajută să gestionăm informația și să comunicăm mai eficient. Totuși, este clar că interesele acestor companii nu sunt doar financiare. Prin produsele lor, urmăresc să-și consolideze poziția pe piață și să influențeze comportamentul consumatorilor în moduri care pot avea efecte pe termen lung asupra societății. Deși ne bucurăm de avantajele

tehnologiei AI, e important să fim conștienți de motivele mai profunde ale acestor companii.

Dacă ne uităm la necesarul uriaș de hardware și energie electrică pentru modele ca GPT-3 Turbo, cu 175 de miliarde de parametri, sau GPT-4, cu 1 trilion de parametri, modele care sunt disponibile publicului general, este greu de crezut că o companie ca OpenAI nu operează pe pierdere. Din aceste observații, putem concluziona că adevăratul câștig pentru aceste companii nu vine din vânzarea directă a produselor, ci din datele colectate de la utilizatori. Aceste date sunt foarte valoroase pentru antrenarea modelelor viitoare, ajutându-le să rămână în fruntea cursei AI.

Soluția prezentată în această lucrare este un asistent personal cu funcționalități similare celor ale asistenților cunoscuți precum Alexa de la Amazon sau Siri de la Apple. Marele avantaj al acestui asistent este că nu rulează pe cloud, ca majoritatea produselor AI avansate, și nu colectează datele utilizatorilor. Mai mult, este accesibil pentru utilizatorul obișnuit și nu este restricționat de regulile impuse de producători. Astfel, își poate folosi creativitatea pentru a oferi o experiență plăcută și diferită, în comparație cu uneltele AI moderne la care sunt obișnuiți utilizatorii.

1.2. Prezentarea Generală a Domeniului

Domeniul acestui proiect se află la intersecția a două nișe foarte populare din lumea inteligenței artificiale: modelele de limbaj mari (LLMs) și asistenții personali interactivi. LLM-urile pot funcționa ca asistenți personali destul de eficienți, cu condiția să aibă o configurație corectă, un prompting expresiv și un număr suficient de mare de parametri (peste 100 de miliarde). Problema este că nimeni nu dispune de puterea de calcul și de energia electrică necesare pentru a rula un astfel de asistent personal pe cont propriu, deci soluția ar fi cloud-ul, dar acest lucru sacrifică intimitatea utilizatorului.

Pe de altă parte, dacă ne uităm la nișa asistenților personali AI, întâlnim uneltele clasice oferite de mari corporații, precum Siri, Alexa, Cortana și Bixby. Aceștia sunt foarte buni în ceea ce fac și pot automatiza numeroase sarcini, dar au și câteva limitări comune: lipsa de libertate datorită faptului că sunt blocați în ecosistemul companiei-mamă, funcționarea pe cloud și repetitivitatea conversațiilor robotizate datorate lipsei de spontaneitate.

Cele două lumi, LLM-urile și asistenții personali AI, au fiecare aspecte cheie care, dacă sunt combinate corect, pot rezulta într-un asistent personal care este mai mult decât un simplu ajutor. Acesta poate deveni un prieten, un tovarăș, care nu doar automatizează sarcini, ci oferă și interacțiuni captivante și plăcute prin dialog.

Capitolul 2. Obiectivele Proiectului

În acest capitol, vom detalia obiectivele principale și secundare ale proiectului. Scopul principal este dezvoltarea unui asistent personal robust (lightweight), pe nume Odin, capabil să automatizeze diverse sarcini zilnice, rulând pe un laptop obișnuit sau pe alte dispozitive locale. Un alt obiectiv principal este adăugarea capacității de dialog uman natural și fluent.

Pe lângă aceste obiective principale, în proiect se urmărește și extinderea funcționalităților asistentului cu câteva obiective secundare. Printre acestea se numără capacitatea de a reformula răspunsurile în funcție de starea de spirit a asistentului, influențată de tonul și limbajul utilizatorului, ceea ce va asigura interacțiuni mai personalizate și adaptive. De asemenea, pentru a face interacțiunea cu Odin cât mai memorabilă și convenabilă, se va elimina necesitatea unei tastaturi și a unui ecran pentru comunicare, adăugând module de conversie vorbire-text (Text-to-Speech) și text-vorbire (Speech-to-Text), facilitând astfel comunicarea verbală între utilizator și asistent.

Un aspect important de menționat în contextul dezvoltării asistentului personal Odin este suportul exclusiv pentru limba engleză. Alegerea acestei limbi se bazează pe câteva argumente solide. Limba engleză este limba în care se găsesc cele mai multe și utile resurse pentru dezvoltarea tehnologiilor AI și NLP. Există o abundență de seturi de date, biblioteci, framework-uri și instrumente specializate disponibile în exclusiv limba engleză, ceea ce accelerează considerabil procesul de dezvoltare. Odin este un asistent complex, cu foarte multe componente și funcționalități avansate. Implementarea acestora în limba română ar necesita un efort considerabil în ceea ce privește resursele de timp și financiare, deoarece ar fi necesară adaptarea și antrenarea modelelor pentru a înțelege și a genera corect limbajul natural în română. În locul dezvoltării unui chatbot limitat, care să suporte și limba română, focusul a fost mutat pe construirea unui chatbot capabil să acopere întreg spectrul domeniului de asistenți personali. Prioritizând limba engleză, s-a reușit dezvoltarea unui asistent performant și versatil, capabil să automatizeze sarcini complexe și să ofere interacțiuni naturale și coerente.

2.1. Obiective Principale

Ca prim obiectiv al proiectului, se urmărește **dezvoltarea nucleului** și a bazei pe care va fi construit, treptat, asistentul personal. Un asistent personal trebuie să îndeplinească un set minim de cerințe pentru a fi eficient și util pentru utilizator. Printre aceste cerințe se numără **automatizarea sarcinilor** și executarea unor acțiuni

predefinite, care simplifică și optimizează activitățile zilnice ale utilizatorului.

Asistentul va trebui să fie capabil să **detecteze intenția** din spatele unui mesaj primit de la utilizator, să extragă informații relevante contextual din acesta și să le folosească mai departe în acțiunile prezise sau predefinite pe care le va executa. De exemplu, dacă utilizatorul solicită setarea unei alarme sau verificarea vremii, asistentul trebuie să înțeleagă aceste cerințe, să identifice entitățile relevante precum data și ora, și să execute acțiunea corespunzătoare în mod eficient. Dezvoltarea nucleului implică și **asigurarea unei arhitecturi modulare și scalabile**, care permite adăugarea ulterioară a noi funcționalități și îmbunătățiri fără a compromite performanța sistemului. Astfel, nucleul va trebui să fie robust și flexibil, capabil să integreze diverse module și componente necesare pentru extinderea capacităților asistentului.

Pentru a crea un asistent satisfăcător, care ar putea avea o șansă în cursa tehnologică, al doilea obiectiv principal este extinderea nucleului menționat anterior, adăugând **capacitatea de a purta o conversație umană captivantă** și, cel mai important, **coerentă** cu utilizatorul. Asistentul trebuie să fie capabil să înțeleagă și să răspundă la o varietate largă de întrebări și comentarii, menținând un flux conversațional natural și captivant. La finalul procesului de dezvoltare, Odin trebuie să fie un asistent personal care nu va zice niciodată „nu”, în comparație cu modelele deja disponibile precum ChatGPT care va refuza conversația în cazul în care comanda este împotriva politicii sale impuse de OpenAI. Aceasta înseamnă că, indiferent de complexitatea sau natura întrebării, Odin va oferi întotdeauna un răspuns, contribuind astfel la crearea unei experiențe de utilizare plăcute și interactive.

2.2. Obiective Secundare

Ca obiective secundare, se propune **extinderea nucleului asistentului** cu un **modul de reformulare a răspunsurilor pe baza stării de spirit**. Acest modul va elimina natura repetitivă a răspunsurilor predefinite, oferind o experiență de utilizare mai naturală și plăcută. Starea de spirit a asistentului va fi influențată de limbajul, tonul și intențiile utilizatorului, permițând astfel asistentului să răspundă în moduri variate și adecvate contextului.

De asemenea, este esențială eliminarea necesității unui mouse și a unei tastaturi pentru comunicare, prin adăugarea modulelor **Speech-to-Text** și **Text-to-Speech**. Aceste module vor permite utilizatorilor să interacționeze cu asistentul personal folosind doar comenzi vocale, transformând astfel interacțiunea într-una ușoară, **cât mai apropiată de interacțiunea umană**. Implementarea acestor obiective va asigura o interfață de utilizare mai intuitivă și accesibilă, facilitând **o experiență conversațională fluidă și captivantă**.

Capitolul 3. Studiu Bibliografic

3.1. Asistenți Personali Inteligenți și Recunoașterea Vocală

Literatura despre asistenții personali inteligenți subliniază importanța integrării recunoașterii vocale avansate și a interfețelor de utilizator prietenoase. Un exemplu proeminent este sistemul SIRI de la Apple, care a revoluționat modul în care utilizatorii interacționează cu dispozitivele mobile folosind comenzi vocale. SIRI, împreună cu Google Voice Search, a stabilit un standard pentru asistenții personali care necesită conexiune la internet pentru a funcționa eficient. Totuși, sistemele mai recente, cum ar fi PARI (Personal Assistant with Voice Recognition Intelligence), dezvoltate de Dr. Kshama V. Kulhalli și colaboratorii săi, sunt capabile să funcționeze și offline, eliminând dependența de conexiunea la internet pentru recunoașterea vocală [1]. Recunoașterea vocală a avut mai multe valuri de inovație majoră, iar sistemele moderne utilizează modele acustice bazate pe LSTM (Long Short-Term Memory) și tehnici de comprimare pentru a rula eficient pe dispozitive mobile. De exemplu, modelele acustice bazate pe CTC (Connectionist Temporal Classification) și compresia SVD (Singular Value Decomposition) permit recunoașterea rapidă și precisă pe smartphone-uri moderne, oferind performanțe în timp real fără a compromite acuratețea [1].

3.2. Încrederea Utilizatorilor și Securitatea Asistenților Personali Inteligenți

Un aspect critic în dezvoltarea asistenților personali inteligenți este încrederea utilizatorilor și securitatea datelor. Conform unui studiu sistematic realizat de Zierau și colaboratorii săi, încrederea în asistenții personali inteligenți (SPA) influențează semnificativ adoptarea și utilizarea acestora. În contextul SPA-urilor, încrederea este definită ca fiind disponibilitatea utilizatorilor de a fi vulnerabili în fața acțiunilor unui sistem autonom, bazându-se pe așteptarea că sistemul va acționa în interesul lor, chiar și în lipsa capacității de a monitoriza sau controla sistemul [2].

Literatura identifică trei perspective principale care constituie cercetarea în domeniul încrederii în SPA-uri: încrederea bazată pe interfața utilizatorului, încrederea bazată pe interacțiune și încrederea bazată pe explicații. Prima categorie se concentrează pe designul static al asistentului, cum ar fi reprezentările audio-vizuale și haptice, pentru a spori încrederea utilizatorilor. A doua categorie se axează pe modul în care interacțiunile dintre utilizator și asistent pe o linie temporală influențează încrederea. În final, încrederea bazată pe explicații explorează modul în care transparența și explicabilitatea acțiunilor asistentului afectează percepțiile utilizatorilor

[2]. O preocupare majoră în utilizarea asistenților personali inteligenți este securitatea și confidențialitatea datelor. Într-un studiu realizat de Cheng și Roedig, aceștia au subliniat că PVAs (Personal Voice Assistants) sunt din ce în ce mai utilizate ca interfețe pentru mediile digitale, dar această creștere aduce și preocupări semnificative legate de securitate și confidențialitate [3]. Studiul lor a evidențiat patru categorii principale de amenințări: controlul accesului, atacurile DoS acustice, confidențialitatea vocală și senzorii acustici. Controlul accesului implică autentificarea utilizatorilor și prevenirea injectării de comenzi vocale neautorizate. Atacurile DoS acustice pot dezactiva temporar interfața vocală prin blocarea canalului acustic. Confidențialitatea vocală se referă la riscurile de pierdere a intimității utilizatorilor din cauza înregistrărilor vocale, iar senzorii acustici pot fi utilizați pentru a extrage informații relevante din mediul utilizatorului .

3.3. Integrarea Inteligenței Artificiale și Învățării Automate

Evoluțiile recente în inteligența artificială și învățarea automată au permis dezvoltarea de asistenți personali mai inteligenți și mai adaptabili. Amazon Echo, Google Assistant și Apple Siri sunt exemple de astfel de sisteme care utilizează tehnici avansate de procesare a limbajului natural (NLP) și învățare automată pentru a oferi asistență utilizatorilor într-o varietate de scenarii de utilizare. Aceste sisteme sunt capabile să efectueze sarcini de la comenzi zilnice, cum ar fi comandarea de produse, până la sarcini specializate, cum ar fi gestionarea cheltuielilor [4]. SPAs manifestă caracteristici de sisteme interactive și inteligente, interacționând cu utilizatorii într-un mod tot mai asemănător comunicării interpersonale. Aceasta include stabilirea de legături emoționale și asigurarea autonomiei în realizarea sarcinilor. Cu toate acestea, natura invazivă a tehnologiilor și complexitatea lor ridicată ridică întrebări legate de responsabilitate și securitatea datelor [4].

3.4. Framework-ul Rasa

Rasa este un framework open-source pentru înțelegerea limbajului natural (NLU) și managementul dialogurilor (DM), destinat dezvoltării software de conversație bazat pe învățare automată. Scopul principal al acestui framework este de a face tehnologiile de gestionare a dialogurilor și înțelegere a limbajului accesibile dezvoltatorilor nespecialiști. Rasa este compus din două componente principale: Rasa NLU și Rasa Core [5].

3.4.1. Rasa NLU

Rasa NLU este modulul responsabil pentru înțelegerea limbajului natural. Acesta include module cuplate liber, care combină mai multe biblioteci de procesare a

limbajului natural și învățare automată într-un API consistent. Principalele funcționalități ale Rasa NLU includ tokenizarea textului, etichetarea părților de vorbire (POS tagging) și recunoașterea entităților folosind câmpuri condiționale aleatorii (CRF) [5]. De asemenea, Rasa NLU permite personalizarea pipeline-urilor de procesare pentru a se potrivi cu nevoile specifice ale proiectului, folosind embeddings pre-antrenate precum GloVe sau embeddings specifice domeniului [5].

3.4.2. Rasa Core

Rasa Core gestionează fluxul conversațional și deciziile în cadrul unui dialog. Acesta utilizează un obiect tracker pentru a stoca starea dialogului, inclusiv sloturile, log-ul tuturor evenimentelor care au avut loc în cadrul unei conversații și acțiunile trecute. Gestionarea dialogului este tratată ca o problemă de clasificare, unde Rasa Core prezice următoarea acțiune de luat pe baza stării curente a tracker-ului [5]. Rasa Core folosește politici (policies) care sunt instanțiate împreună cu un featurizer pentru a crea o reprezentare vectorială a stării curente a dialogului.

3.4.3. Flexibilitatea și Utilitatea Rasa

Unul dintre principalele avantaje ale Rasa este modularitatea sa, care permite integrarea ușoară cu alte sisteme. De exemplu, Rasa Core poate fi utilizat ca manager de dialog în combinație cu servicii NLU diferite de Rasa NLU [5]. Acest lucru permite reutilizarea modelelor de dialog antrenate pe diverse limbi și proiecte, oferind o flexibilitate ridicată dezvoltatorilor.

3.5. Transformers

Transformerele sunt o arhitectură de învățare profundă care a revoluționat domeniul procesării limbajului natural (NLP). Introduși de către cercetătorii Google în lucrarea de referință [6] „Attention Is All You Need” de Vaswani et al. în 2017, transformerele se bazează în întregime pe mecanismele de atenție, renunțând la recurență și convoluții. Această arhitectură permite o paralelizare semnificativă, făcând-o extrem de eficientă pentru antrenarea pe seturi de date mari. Transformerele constau într-o structură encoder-decoder în care atât encoderul, cât și decoderul sunt compuse din straturi de auto-atenție și rețele neuronale feed-forward. Mecanismul de auto-atenție permite modelului să cântărească importanța diferitelor cuvinte într-o propoziție, oferind o metodă robustă pentru captarea dependențelor, indiferent de distanța lor în secvență. Această abordare a demonstrat performanțe superioare în sarcini precum traducerea automată, obținând rezultate de vârf pe diverse repere.

3.6. Modele de Limbaj de Mari Dimensiuni (LLMs)

Modelele de limbaj de mari dimensiuni (LLMs) reprezintă o evoluție majoră în domeniul procesării limbajului natural (NLP). Aceste modele sunt construite pe baza arhitecturii Transformer și sunt antrenate pe corpuri de text masive. Un exemplu notabil este GPT-3, care conține 175 de miliarde de parametri și a demonstrat capacități impresionante în învățarea în context și generarea de text [7].

În cadrul lucrării curente, modelul GPT-Neo cu 1.8 miliarde de parametri a fost ales pentru a oferi răspunsuri detaliate și naturale. GPT-Neo este un model open-source care a fost dezvoltat pentru a reproduce performanțele GPT-3, dar cu o arhitectură accesibilă comunității de cercetare [7]. Acest model permite o generare rapidă de text, fiind optim pentru a rula pe hardware limitat, cum ar fi o placă video cu 6GB VRAM.

3.6.1. Fine-tuning

Fine-tuning-ul, sau reglarea fină, este un proces esențial pentru adaptarea modelelor LLM la cazuri specifice de utilizare. Acest proces implică antrenarea modelului pe un set de date relevant pentru aplicația țintă, permițând modelului să învețe nuanțele și particularitățile domeniului specific. Fine-tuning-ul îmbunătățește performanța modelului în sarcini specializate și asigură că răspunsurile generate sunt precise și contextuale [7]. Fine-tuning-ul permite modelelor LLM mai mici, care au între 1 și 5 miliarde de parametri, să atingă performanțe comparabile cu modelele mult mai mari pe cazuri specifice de utilizare. Acest lucru este esențial pentru utilizatorii individuali sau pentru organizațiile cu resurse hardware limitate, deoarece permite utilizarea eficientă a resurselor disponibile fără a compromite calitatea rezultatelor. Modele precum GPT-Neo, care au aproximativ 1.8 miliarde de parametri, pot fi ajustate fin pentru a oferi performanțe remarcabile pe aplicații specifice, cu un cost computațional relativ scăzut [7].

3.7. Detectarea Intențiilor și Completarea Sloturilor

Detectarea intențiilor și completarea sloturilor sunt două sarcini esențiale pentru construirea unui sistem de înțelegere a limbajului vorbit (SLU). Modelul propus de Yu Wang și colegii săi introduce o nouă structură RNN Bi-model care îmbunătățește performanța prin luarea în considerare a impactului transversal între cele două sarcini [8]. Acest model utilizează două LSTM-uri bidirecționale (BLSTM) corelate pentru a detecta intențiile și a completa sloturile în mod concurrent pentru fiecare enunț.

Studiile anterioare au tratat detectarea intențiilor și completarea sloturilor ca două sarcini separate sau au utilizat un model secvență-la-secvență (sequence-to-sequence) pentru a genera simultan etichete semantice și intenții [8]. În schimb, structura Bi-model propusă de Wang et al. utilizează două rețele BLSTM

interconectate, fiecare responsabilă de una dintre cele două sarcini, îmbunătățind astfel performanța prin partajarea stărilor interne ale rețelelor. Experimentele efectuate pe setul de date ATIS au arătat că structura Bi-model cu un decodor a obținut performanțe de ultimă generație, cu o îmbunătățire de aproximativ 0.5% în acuratețea intențiilor și 0.9% în completarea sloturilor [8]. În plus, modelul Bi cu decodor a depășit performanța modelului fără decodor, demonstrând beneficiile complexității suplimentare introduse de decodor.

O altă arhitectură care este folosită și în lucrarea curentă este arhitectura DIET (Dual Intent and Entity Transformer), o soluție avansată pentru sarcinile de clasificare a intențiilor și recunoașterea entităților, prezentată de Tanja Bunk și colegii săi [9]. DIET este un model multi-task care combină avantajele reprezentărilor dense pre-antrenate cu caracteristicile sparse la nivel de cuvânt și n-gram de caractere. DIET utilizează un transformator cu două straturi și atenție relativă la poziție pentru a codifica contextul întregii propoziții [9]. Modelul poate funcționa eficient chiar și fără utilizarea de embeddings pre-antrenate, folosind doar caracteristici sparse. Totuși, adăugarea de embeddings pre-antrenate precum ConveRT, BERT sau GloVe îmbunătățește performanța generală a modelului. Pe setul de date NLU-Benchmark, DIET a demonstrat performanțe competitive, depășind modelele existente în ceea ce privește acuratețea și viteza de antrenare [9]. Utilizarea unui strat CRF (Conditional Random Field) pentru recunoașterea entităților și a unei funcții de pierdere bazată pe produsul scalar pentru clasificarea intențiilor contribuie la eficiența și precizia modelului.

3.8. Sisteme Text-to-Speech (TTS)

Sistemele Text-to-Speech (TTS) au înregistrat progrese remarcabile datorită avansurilor în învățarea profundă. Aceste sisteme permit transformarea textului în vorbire sintetică de înaltă calitate, utilizând modele complexe de rețele neuronale. În această secțiune, vom discuta despre două modele de ultimă generație: NaturalSpeech și XTTS.

NaturalSpeech este un sistem TTS revoluționar care a obținut rezultate de ultimă generație în sintetizarea vorbirii. Acest model se bazează pe o arhitectură inovatoare și introduce mai multe tehnici avansate pentru a îmbunătăți calitatea și naturalitatea vorbirii sintetizate [10]. NaturalSpeech utilizează o abordare numită „prosody modeling” pentru a captura intonația și ritmul vorbirii naturale. Modelul folosește un encoder-decoder cu atenție pentru a mapa secvențele de text în reprezentări acustice, care sunt apoi transformate în vorbire folosind un vocoder avansat. Un aspect cheie al NaturalSpeech este abilitatea sa de a genera vorbire expresivă și realistă, chiar și în cazurile în care textul de intrare este ambiguu sau incomplet. Experimentele au arătat că NaturalSpeech depășește alte modele de referință în ceea ce privește naturalitatea și claritatea vorbirii sintetizate, oferind performanțe superioare în diverse limbi și stiluri de vorbire. Acest model este deosebit de eficient în capturarea variațiilor subtile ale

prosodie, făcându-l ideal pentru aplicații care necesită vorbire expresivă și adaptabilă [10].

Un alt model performant, de data asta folosit în lucrare este XTTS, un model multilingv ZS-TTS (zero-shot multi-speaker TTS) care sprijină limbile cu resurse reduse și medii. Modelul utilizează un VQ-VAE (Vector Quantised-Variational AutoEncoder) cu 13 milioane de parametri pentru a codifica fiecare cadru al mel-spectrogramului. Encoder-ul GPT-2, care conține 443 milioane de parametri, primește tokeni de text și prezice codurile audio VQ-VAE. Decoder-ul, bazat pe vocoderul HiFi-GAN, este responsabil de generarea finală a vorbirii sintetice [11]. Experimentele au demonstrat că XTTS obține performanțe remarcabile în ceea ce privește similaritatea vorbitorului și naturalitatea vocii sintetizate, depășind alte modele de referință precum StyleTTS 2, Tortoise, YourTTS și HierSpeech++ [11]. De asemenea, XTTS este capabil să efectueze TTS încrucișat între limbi, producând un accent nativ în limba țintă, fără a necesita un set de date de antrenament paralel. XTTS poate fi ajustat fin pentru a se adapta la noi vorbitori sau condiții de înregistrare diferite, utilizând mostre scurte de vorbire. Experimentele au arătat că fine-tuning-ul modelului cu doar 10 minute de vorbire a îmbunătățit semnificativ similaritatea vorbitorului în setările de transfer între vorbitori [11]. Acest proces de fine-tuning este esențial pentru adaptarea modelului la aplicații specifice și pentru a asigura că răspunsurile generate sunt precise și contextuale. XTTS oferă și capabilități avansate de clonare a vocii, permițând generarea de vorbire sintetică care imită stilul și prosodia unui vorbitor dat. Acest lucru este realizat prin antrenarea modelului cu date vocale din multiple limbi și prin utilizarea unei arhitecturi eficiente de rețea neuronală care combină atenția scalată cu percepția resampler [11].

Capitolul 4. Analiză și Fundamentare Teoretică

Scopul principal al acestui capitol este de a prezenta arhitectura conceptuală a sistemului Odin, precum și de a descrie algoritmi propuși, protocoalele utilizate și modelele abstracte care constituie structura logică a asistentului personal. Pe parcursul acestui capitol, se vor discuta în detaliu componentele fundamentale ale sistemului distribuit care definește asistentul personal, precum și protocoalele și tehnicile de comunicare dintre acestea, având ca scop aprofundarea unui pipeline bine definit de procesare a comenzilor.

Acest capitol va explica soluția propusă din punct de vedere teoretic, demonstrând proprietățile și valoarea teoretică a acesteia prin prezentarea algoritmilor de înțelegere a limbajului natural (eg. Natural Language Understanding, NLU), detectare a intențiilor, completare a sloturilor și generare a dialogului. Vor fi explicate protocoalele de comunicare între componentele distribuite ale sistemului. De asemenea, se vor prezenta modelele conceptuale și arhitecturile utilizate pentru implementarea funcționalităților de bază ale asistentului, inclusiv modelele de limbaj mare (eg. Large Language Models, LLM) pentru generarea și reformularea dialogului și modelele de conversie text-voce (eg. Text To Speech, TTS).

În acest capitol se vor justifica alegerile soluțiilor arhitecturale utilizate în dezvoltarea lui Odin, evidențiind avantajele și dezavantajele acestora. De asemenea, se va descrie la un nivel abstract fluxul de procesare a comenzilor, de la intrarea utilizatorului până la generarea răspunsurilor vocale. Scopul este de a oferi o înțelegere aprofundată a principiilor funcționale care stau la baza asistentului personal Odin și a modului în care acestea contribuie la performanța și fiabilitatea sistemului.

4.1. Arhitectura Conceptuala

Cu scopul de a înțelege și de a intra în detaliu în componentele definitorii ale sistemului, mai întâi trebuie să privim sistemul de la un nivel înalt, mai abstract, și să urmărim fluxul de informații de-a lungul componentelor. Toate modulele prezentate în **Figura 1** sunt esențiale în funcționarea optimă a asistentului.

La un nivel înalt, arhitectura conceptuală a sistemului Odin este compusă din mai multe blocuri interconectate, fiecare având un rol specific în procesarea comenzilor și generarea răspunsurilor. În centrul acestei arhitecturi se află nucleul, responsabil pentru înțelegerea limbajului natural și gestionarea fluxului conversațional. Nucleul comunică cu diverse componente externe prin intermediul unor protocoale standardizate, pentru a asigura un flux de informații coerent și eficient.

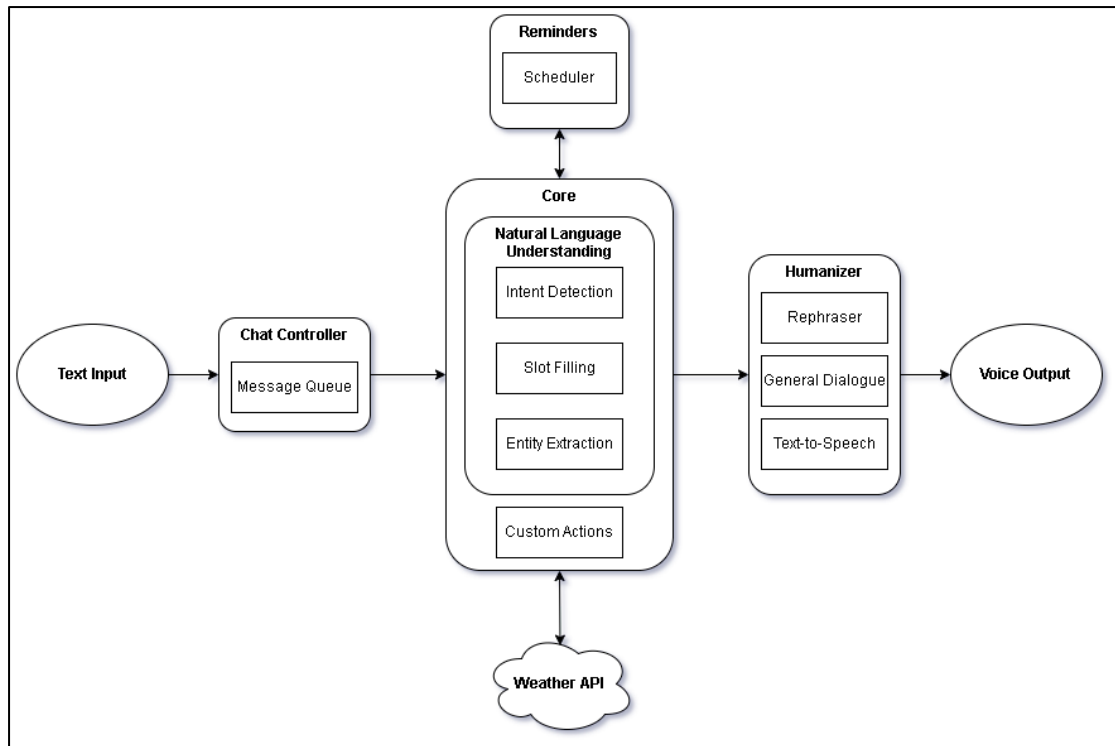


Figura 1: Arhitectura Conceptuală

4.2. Înțelegerea Limbajului Natural

Înțelegerea limbajului natural (eg. Natural Language Understanding, NLU) este o ramură esențială a inteligenței artificiale, care se ocupă cu interpretarea și înțelegerea limbajului uman de către computere. Spre deosebire de oameni, care pot înțelege contextul și semnificația cuvintelor într-un mod intuitiv, un program de calculator nu poate procesa limbajul natural în mod direct. Din acest motiv, este necesar să se utilizeze algoritmi și procese matematice complexe pentru a transforma textul din cuvinte în reprezentări algebrice, pe care computerele le pot manipula și înțelege.

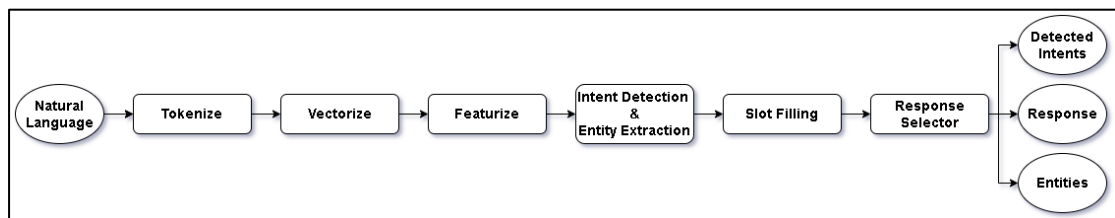


Figura 2: Pipeline-ul NLU

Fiecare asistent personal inteligent are obligatoriu la baza o formă a pipeline-ului NLU. În **Figura 2** este evidențiat la modul general și conceptual un pipeline de înțelegere a limbajului natural, acesta poate prezenta mici schimbări la implementare, în funcție de preferințele dezvoltatorului. Pipeline-ul implică mai multe etape critice, fiecare

contribuind la transformarea textului într-o formă pe care sistemul o poate utiliza pentru a lua decizii și a genera răspunsuri adecvate.

4.2.1. Tokenizarea

Tokenizarea este prima etapă a analizei limbajului natural și este definită ca fiind procesul de fragmentare a textului într-o serie de unități mai mici, cunoscute sub numele de tokeni. Un token poate fi fie un caracter unic, un cuvânt sau o frază, în funcție de gradul de detaliere al tokenizării. Scopul general al tokenizării este acela de a produce unități ușor de procesat. Există mai multe forme de tokenizare, fiecare având avantajele și dezavantajele sale. Cea mai comună formă de tokenizare este *tokenizarea pe cuvinte*, adică tokenizarea textului la fiecare spațiu dintre cuvinte. Această formă de tokenizare este mai ales potrivită pentru limbile în care cuvintele sunt bine separate de spații, cum ar fi limba engleză. În schimb, în limbile în care cuvintele nu sunt clar delimitate, această metodă poate fi mai puțin eficientă.

Tokenizarea pe caractere înseamnă împărțirea textului în caractere individuale. Această tehnică de tokenizare este aplicabilă în limbile cu alfabet mare sau în cazul în care este necesară o analiză detaliată la nivel de caracter. Însă, acest lucru poate conduce la costuri ridicate ale complexității și datelor prelucrate.

Tokenizarea pe sub-cuvinte combină cele două abordări discutate anterior și este utilizată într-o măsură considerabilă în modelele moderne de limbaj, cum ar fi BERT sau GPT. Procesul constă în descompunerea unui cuvânt în sub-cuvinte, cum ar fi prefixe, sufixe sau alte grupuri de caractere care au o semnificație. Acest lucru permite procesarea eficientă a cuvintelor necunoscute sau rare și reducerea dimensiunii vocabularului necesar.

Algoritmii de tokenizare, cum ar fi Byte Pair Encoding (BPE) sau WordPiece, sunt utilizați pentru a efectua tokenizarea pe sub-cuvinte. Acești algoritmi funcționează prin iterarea asupra unui corp de text și identificarea celor mai frecvente perechi de caractere sau sub-cuvinte, care sunt apoi combinate într-un token. Acest proces continuă până când se atinge o dimensiune predefinită a vocabularului.

4.2.2. Vectorizarea

Vectorizarea este un pas crucial în procesul de înțelegere a limbajului natural (NLU), fiind etapa în care textul este transformat în reprezentări numerice numite vectori. Acești vectori sunt folosiți de algoritmii de învățare automată pentru a face predicții și a interpreta datele textuale. Practic, vectorii sunt structuri de date de dimensiune fixă care conțin informații numerice despre text. Dimensiunea unui vector poate varia în funcție de metoda utilizată pentru vectorizare și de mărimea vocabularului.

Există diverse metode de vectorizare, fiecare cu avantajele și dezavantajele sale. Una dintre cele mai simple metode este bag-of-words, care creează un vector binar sau ponderat pentru fiecare document. Fiecare element al vectorului reprezintă prezența sau frecvența unui cuvânt specific din vocabular. De exemplu, dacă avem propoziția "The cat saw the man", vom obține un vocabular ['the', 'cat', 'saw', 'man'] și o reprezentare bag-of-words [2, 1, 1, 1]. Deși această metodă este simplă și ușor de implementat, ea nu capturează relațiile semantice sau contextuale dintre cuvinte.

Metodele moderne de vectorizare, cum ar fi Word2Vec, GloVe și FastText, transformă cuvintele în vectori de dimensiune fixă, unde cuvintele cu semnificații similare sunt reprezentate prin vectori asemănători. Aceste metode se bazează pe rețele neuronale și sunt antrenate pe corpuri mari de text pentru a învăța reprezentările contextuale ale cuvintelor. De exemplu, Word2Vec utilizează două abordări principale: modelul continuous bag-of-words (CBOW) și modelul skip-gram. Modelul CBOW prezice un cuvânt dat contextul său, în timp ce modelul skip-gram prezice contextul dat un cuvânt.

Un aspect interesant al vectorilor de cuvinte este posibilitatea efectuării de operații aritmetice asupra lor. De exemplu, bine cunoscutul exemplu "king - man + woman = queen" demonstrează că relațiile semantice dintre cuvinte pot fi capturate și manipulate prin vectori. Aceasta înseamnă că putem folosi vectorii pentru a rezolva analogii și pentru a descoperi relații ascunse între cuvinte.

4.2.3. Caracterizarea (Featurizing)

Featurizarea (caracterizarea) este un alt proces similar și foarte strâns legat de vectorizare în scopul de a extrage informații din limbajul natural (NLU). Ambele procese transformă textul brut în reprezentări numerice care pot fi folosite de algoritmi de învățare automată. Featurizarea, în contrast cu vectorizarea, se concentrează mai mult pe extragerea trăsăturilor de domeniu ale textului. Deși, de obicei, doar una dintre aceste două etape este necesară, când vorbim despre proiectarea conceptuală a sistemului, vom menționa ambele etape pentru a ilustra rolurile diferite și complementare ale fiecăreia dintre ele în pipeline-ul ideal de învățare a limbajului.

Featurizarea oferă, de asemenea, un mecanism pentru a localiza și utiliza caracteristici lexicale și sintactice ale textului care sunt relevante pentru sarcina de învățare automată. De exemplu, utilizând featurizarea, putem redimensiona cuvintele comune în funcție de importanță utilizând tehnici TF-IDF (frecvență termenului-frecvență inversă a documentului), ceea ce ajută la reducerea influenței cuvintelor frecvente, dar nesemnificative.

4.2.4. Detectia Intențiilor & Extragerea Entităților

Detectarea intențiilor și extragerea entităților sunt două procese esențiale în înțelegerea limbajului natural (NLU), fiind de obicei tratate împreună datorită interdependenței lor. Aceste etape sunt fundamentale pentru ca un asistent personal să poată înțelege corect și să răspundă la comenzile utilizatorilor.

Detectarea intențiilor este modalitatea prin care un sistem NLU identifică intenția sau scopul din spatele mesajului utilizatorului. De exemplu, dacă un utilizator spune „*I want to book a table*”, intenția poate fi identificată ca *book_table*. Aceasta implică clasificarea mesajului într-una dintre multiplele intenții predefinite, bazându-se pe caracteristicile extrase anterior.

Extragerea entităților este procesul de identificare și extragere a unor informații specifice din text, care au o semnificație particulară. În exemplul de mai sus, entitatea ar putea fi „*table*”, iar dacă mesajul ar fi „*I want to book a table at 9 PM*”, entitățile suplimentare ar fi „*9 PM*”. Aceste entități sunt date structurale extrase din text pentru a fi utilizate în procesarea ulterioară.

Majoritatea algoritmilor tratează detectarea intențiilor și extragerea entităților împreună deoarece aceste două sarcini sunt strâns legate. Intenția unui mesaj poate influența tipurile de entități care sunt relevante și invers. De exemplu, într-un mesaj de tip „*rezervare*”, entități precum data, ora și locația ar fi relevante. Tratarea acestor sarcini împreună permite algoritmilor să utilizeze informațiile contextuale pentru a face predicții mai precise.

DIET (Dual Intent and Entity Transformer) este un exemplu de model care efectuează atât detectarea intențiilor, cât și extragerea entităților simultan. DIET este un model multi-task care folosește o arhitectură de transformator, discutată în detaliu la capitolul 4.3.1, pentru a învăța reprezentări comune pentru ambele sarcini. Modelul utilizează un transformator pentru a procesa acești vectori, generând reprezentări contextuale pentru fiecare token din mesaj. Aceste reprezentări capturează atât informațiile locale, cât și cele globale din secvență. Pentru predicția intenției, DIET agregă reprezentările întregului mesaj și le trece printr-un strat de clasificare, identificând astfel intenția dintr-un set predefinit.

Pentru extracția entităților, DIET folosește reprezentările contextuale ale tokenilor individuali, clasificând fiecare token în funcție de entitățile posibile sau marcându-l ca nefiind o entitate. Acest proces se realizează printr-un strat de Conditional Random Field (CRF), care ia în considerare dependențele dintre tokenii adiacenți pentru a face predicții coerente.

DIET este antrenat end-to-end pe ambele sarcini simultan, optimizând o funcție de pierdere combinată pentru detectarea intențiilor și extracția entităților. Aceasta permite modelului să beneficieze de informațiile partajate între cele două sarcini, învățând reprezentări mai bogate și mai precise.

4.2.5. Completarea Parametrilor Intenției (Slot Filling)

Completarea sloturilor este un pas strâns legat de cel anterior cu detecția intențiilor și extracția entităților, fiind adesea considerat o extensie a acestor pași. De fapt, sub-pașii din detecția intențiilor și extracția entităților constituie în mare parte procesul de slot filling. Totuși, pentru claritatea arhitecturii conceptuale și pentru a descrie un pipeline NLU ideal și complet, vom trata slot filling ca un pas suplimentar.

Slot filling este procesul prin care informațiile extrase din text sunt plasate în sloturi predefinite necesare pentru completarea unei anumite acțiuni sau intenții. Practic, după ce intenția și entitățile unui mesaj sunt detectate, slot filling organizează aceste informații într-un format structurat, ușor de utilizat de către sistemul de procesare ulterioară.

4.2.6. Selecția Răspunsului

Selecția răspunsului (response selection) este ultimul pas în pipeline-ul de înțelegere a limbajului natural (NLU). Acest pas nu este neapărat un proces de înțelegere a limbajului, ci mai degrabă o tranziție de la înțelegere la acțiune, dar este esențial când vine vorba de un asistent personal inteligent. Este momentul în care sistemul, pe baza înțelegerii sale anterioare a mesajului utilizatorului, decide ce răspuns sau acțiune să efectueze.

Selecția răspunsului este procesul prin care un model NLU alege cel mai potrivit răspuns sau acțiune în urma analizei mesajului utilizatorului. Aceasta se bazează pe intențiile și entitățile identificate anterior, organizate în sloturi, și implică fie aplicarea unor reguli predefinite, fie utilizarea modelelor de predicție pentru a determina acțiunea corespunzătoare.

4.3. Modele Mari de Limbaj

Modelele de limbaj mari (eg. Large Language Models, LLMs) sunt algoritmi avansați de procesare a limbajului natural (NLP) care utilizează rețele neuronale de mare complexitate pentru a înțelege, genera și manipula textul uman. Aceste modele sunt antrenate pe seturi masive de date textuale și au un număr foarte mare de parametri, ceea ce le permite să capteze și să reproducă subtilitățile și diversitatea limbajului uman. Exemple notabile de LLMs includ GPT-3, BERT, RoBERTa și T5. Aceste modele sunt folosite în diverse aplicații, cum ar fi chatboți, traducere automată, completare automată de text, și multe altele.

4.3.1. Arhitectura Transformer

Transformers [6] reprezintă un tip de arhitectură de învățare profundă (eg. deep learning) care a revoluționat domeniul procesării limbajului natural (NLP) în ultimii ani. Sunt utilizate pe scară largă pentru sarcini precum traducerea limbajului, clasificarea textului, analiza sentimentelor și multe altele. Arhitectura transformer, introdusă în 2017 de către cercetătorii de la Google, propune o abordare nouă pentru sarcinile de NLP, bazată exclusiv pe mecanismul de autoatenție (self-attention), un tip de mecanism de atenție care permite modelului să evalueze importanța diferitelor cuvinte într-o propoziție atunci când le encodează într-o reprezentare vectorială de dimensiune fixă.

Arhitectura transformer este compusă dintr-un encoder și un decoder, fiecare fiind alcătuit din multiple straturi de autoatenție și rețele neuronale feedforward. Mecanismul de autoatenție este nucleul transformerului, permițând modelului să evalueze importanța diferitelor cuvinte într-o propoziție bazându-se pe afinitatea lor. În plus, transformerul introduce biasul pozițional, care permite modelului să țină evidența pozițiilor relative ale cuvintelor într-o propoziție, aspect esențial deoarece ordinea cuvintelor poate afecta semnificativ sensul propoziției.

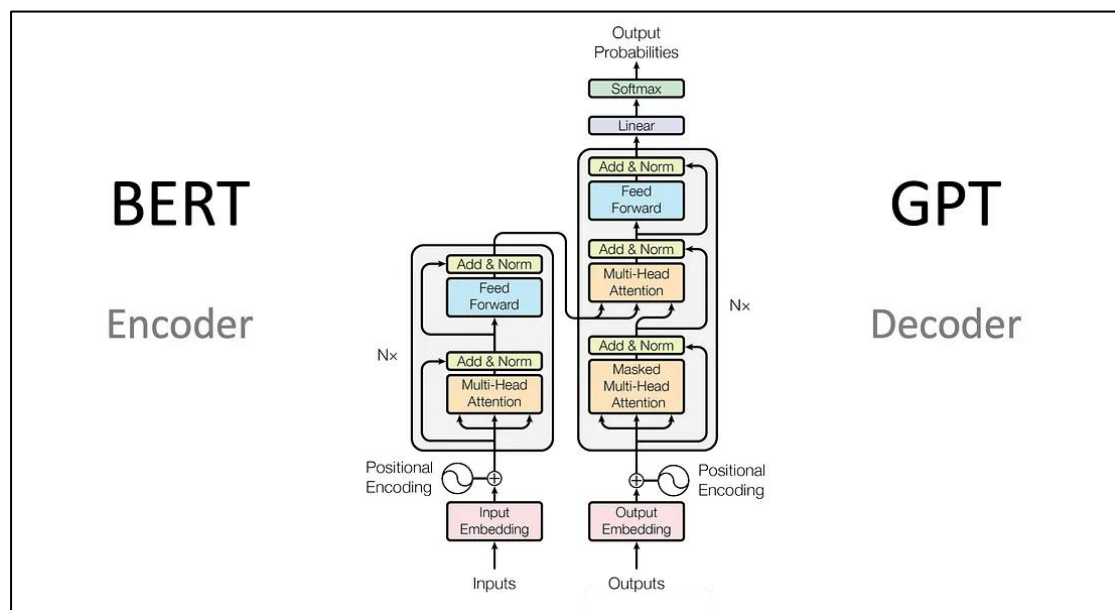


Figura 3: Arhitectura Transformer

Arhitectura encoder-decoder a transformerului este utilizată pentru sarcini precum traducerea limbajului, unde modelul trebuie să preia o propoziție într-o limbă și să producă o propoziție într-o altă limbă. Encoderul preia propoziția de intrare și produce o reprezentare vectorială de dimensiune fixă, care este apoi alimentată în decoder pentru a genera propoziția de ieșire. Decoderul folosește atât autoatenție, cât și cross-attention, unde mecanismul de atenție este aplicat atât asupra ieșirii encoderului, cât și asupra intrării decoderului. Unul dintre cele mai populare modele encoder-decoder de transformer este T5 (Text-to-Text Transfer Transformer) [7], introdus de

Google în 2019. T5 poate fi ajustat pentru o gamă largă de sarcini NLP, inclusiv traducerea limbajului, răspunsul la întrebări, sumarizarea textului și multe altele.

Arhitectura transformer encoder este utilizată pentru sarcini precum clasificarea textului, unde modelul trebuie să clasifice un text într-una din mai multe categorii predefinite, cum ar fi analiza sentimentelor, clasificarea tematică sau detectarea spamului. Encoderul preia o secvență de tokeni și produce o reprezentare vectorială de dimensiune fixă a întregii secvențe, care poate fi apoi utilizată pentru clasificare. Unul dintre cele mai populare modele encoder de transformer este BERT (Bidirectional Encoder Representations from Transformers) [7], introdus de Google în 2018. BERT este preantrenat pe cantități mari de date textuale și poate fi ajustat pentru o gamă largă de sarcini NLP. Spre deosebire de arhitectura encoder-decoder, encoderul transformer este preocupat doar de secvența de intrare și nu generează nicio secvență de ieșire.

Arhitectura transformer decoder este utilizată pentru sarcini precum generarea limbajului, unde modelul trebuie să genereze o secvență de cuvinte pe baza unui prompt sau context de intrare. Decoderul preia o reprezentare vectorială de dimensiune fixă a contextului și o folosește pentru a genera o secvență de cuvinte unul câte unul, fiecare cuvânt fiind condiționat de cuvintele generate anterior. Unul dintre cele mai populare modele decoder de transformer este GPT-3 (Generative Pre-trained Transformer 3) [7], introdus de OpenAI în 2020. GPT-3 este un model de limbaj masiv care poate genera text asemănător celui uman într-o gamă largă de stiluri și genuri. Arhitectura transformer decoder introduce o tehnică numită triangle masking pentru atenție, care asigură că mecanismul de atenție se uită doar la tokenii din stânga tokenului curent generat, prevenind astfel modelul de la “trișare” uitându-se la tokeni pe care încă nu i-a generat.

Arhitectura transformer, deși revoluționară în domeniul NLP, vine cu propriile sale dezavantaje semnificative. Unul dintre principalele obstacole ale acestei arhitecturi este resursa intensivă pe care o necesită, atât în termeni de hardware, cât și de energie pentru generare și antrenare. Modelele de limbaj mari (LLMs) sunt adesea foarte greu de rulat local din cauza cerințelor lor ridicate de resurse, ceea ce face ca majoritatea modelelor fiabile să fie hostate pe cloud de către companiile mari. Modelele transformer, mai ales cele mari și competente, necesită o cantitate enormă de resurse hardware. Antrenarea și rularea acestor modele pe hardware local implică de obicei utilizarea unor plăci grafice de înaltă performanță (GPU-uri) sau unități de procesare tensorială (TPU-uri), care nu sunt accesibile pentru toți utilizatorii. Chiar și modelele mai mici, care pot fi rulate pe GPU-uri comerciale, necesită totuși cantități semnificative de memorie și putere de calcul.

Un alt dezavantaj major al arhitecturii transformer este scalarea cvadratică a memoriei necesare, datorită mecanismului de auto-atenție (self-attention). Mecanismul de auto-atenție permite fiecărui token să fie legat de fiecare alt token din secvență, ceea ce înseamnă că memoria necesară pentru a stoca aceste legături crește proporțional cu pătratul lungimii secvenței de intrare. Acest comportament cvadratic devine

problematic pentru secvențele lungi de text, deoarece necesită din ce în ce mai multă memorie pe măsură ce lungimea secvenței crește.

4.3.2. Integrarea unui LLM într-un Asistent Personal Inteligent

Integrarea unui model de limbaj mare (LLM) într-un asistent personal inteligent reprezintă o provocare semnificativă, mai ales atunci când ținta este accesibilitatea pentru consumatorii de rând care nu dispun de hardware dedicat de înaltă performanță. Majoritatea modelelor LLM necesită resurse computaționale semnificative, inclusiv plăci grafice de ultimă generație, care sunt adesea inaccesibile pentru utilizatorii obișnuiți. Totuși, cu avansurile recente în dezvoltarea modelelor de limbaj, au apărut soluții care permit utilizarea acestor tehnologii avansate pe hardware mai modest, făcându-le accesibile pentru un public mai larg.

Un exemplu notabil de astfel de model este GPT-Neo. Dezvoltat de EleutherAI, GPT-Neo [7] a fost conceput pentru a oferi o alternativă open-source la GPT-3. Acesta aduce puterea modelelor de limbaj mari într-un format mai accesibil și mai ușor de integrat în aplicații de consum. Modelul GPT-Neo cu 1.8 miliarde de parametri utilizează o arhitectură similară cu cea a GPT-3, bazată pe transformer decoder. Scopul dezvoltării GPT-Neo a fost de a democratiza accesul la modele de limbaj mari și de a permite cercetătorilor și dezvoltatorilor să beneficieze de puterea acestor modele fără a depinde de resursele restricționate ale marilor corporații.

4.3.3. Procesul de Reglare Fină

Procesul de reglare fină (fine-tuning) reprezintă un pas esențial în utilizarea eficientă a modelelor de limbaj mari (LLMs) pentru aplicații specifice. Fine-tuning-ul implică ajustarea unui model preantrenat pe un set vast de date generale pentru a-l adapta la un caz de utilizare particular, folosind un set de date specific și adesea mult mai mic. Acest proces îmbunătățește semnificativ performanțele modelului în contextul aplicației vizate, făcându-l mai precis și mai relevant. Procesul consta în adaptarea unui model, deja antrenat pe o gama largă de date, pentru o sarcină specifică prin continuarea antrenamentului său pe un set de date specializat. Acest proces ajustează parametrii modelului pentru a se concentra mai bine pe tipurile de probleme și date specifice aplicației finale. O ilustrație relevantă pentru comportamentul unui model în urma acestui proces este prezentată în **Figura 4**.

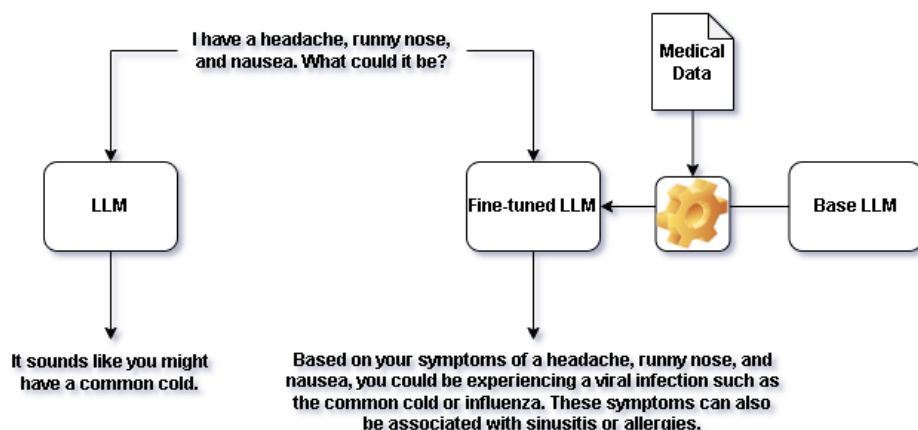


Figura 4: Conceptul de Fine-Tuning

Un model de dimensiuni mai mici poate ajunge foarte aproape de performanțele unui colos precum GPT-3 prin fine-tuning. Deși GPT-3 are 175 de miliarde de parametri, un model mai mic, cum ar fi GPT-Neo cu 1.8 miliarde de parametri, poate fi fin ajustat pentru a se comporta extrem de bine într-un domeniu specific, apropiindu-se de performanțele modelului mai mare. Acest lucru este posibil deoarece fine-tuning-ul permite modelului să învețe și să se specializeze pe un set de date restrâns, devenind astfel foarte eficient în acel context.

4.4. Metrice de Evaluare

Pentru a evalua performanța unui asistent personal bazat pe inteligența artificială, se utilizează o varietate de metrice standard de evaluare. Printre cele mai importante se numără acuratețea, precizia, recall-ul și scorul F1. Aceste metrice oferă o imagine de ansamblu asupra capacității modelului de a clasifica corect intențiile și de a extrage entități relevante din interacțiunile cu utilizatorul. Acuratețea reprezintă proporția predicțiilor corecte din totalul predicțiilor efectuate. Este calculată ca raportul dintre numărul de predicții corecte și numărul total de exemple evaluate. Precizia măsoară acuratețea predicțiilor pozitive ale modelului, fiind raportul dintre numărul de predicții corecte pozitive și numărul total de predicții pozitive (atât corecte, cât și incorecte). Recall-ul (sau Sensibilitatea) indică capacitatea modelului de a identifica corect toate instanțele pozitive, fiind raportul dintre numărul de predicții pozitive corecte și numărul total de instanțe pozitive reale. Scorul F1 este media armonică între precizie și recall, oferind o singură valoare care echilibrează cele două metrice.

Un alt mod eficient de vizualizare a performanței modelelor clasificatoare este utilizarea matricilor de confuzie. O matrice de confuzie este un tabel care permite vizualizarea performanței unui algoritm de clasificare. Fiecare rând al matricei reprezintă instanțele dintr-o clasă actuală, în timp ce fiecare coloană reprezintă

instanțele dintr-o clasă prezisă. Matricea de confuzie oferă o imagine clară a erorilor de clasificare comise de model și permite identificarea precisă a tipurilor de greșeli pe care modelul le face.

Pe lângă aceste metrice standard, este esențială și testarea unitară a componentelor individuale ale sistemului, în special pentru acele componente care nu se bazează pe AI. Testarea unitară asigură că fiecare componentă funcționează corect în mod izolat, contribuind la stabilitatea și fiabilitatea întregului sistem.

Testarea end-to-end reprezintă procesul de evaluare a întregului sistem integrat, de la început până la sfârșit, asigurându-se că toate componentele lucrează împreună așa cum este de așteptat. Un aspect crucial al testării end-to-end este evaluarea experienței utilizatorului (UX), care implică obținerea de feedback de la utilizatori reali pentru a determina cât de satisfăcătoare este interacțiunea cu asistentul. Utilizatorii pot evalua asistentul pe baza unor criterii predefinite, cum ar fi înțelegerea comenzilor, acuratețea răspunsurilor, coerența dialogului și nivelul de umanitate al interacțiunilor.

Capitolul 5. Proiectare de Detaliu și Implementare

Conținutul acestui capitol constă în prezentarea aspectelor detaliate legate de implementarea propriu-zisă a asistentului personal Odin, care dispune de capabilități de dialog și răspunsuri umane la comenzi, bazate pe emoții. În continuare, se prezintă schema generală a sistemului, care aprofundează funcționarea tuturor componentelor distribuite din care este constituit bot-ul și interacțiunile dintre acestea. Modulele prezentate vor fi detaliate, incluzând informații importante despre implementare, rolurile fiecăruia, problemele întâmpinate și soluțiile sau compromisurile găsite.

Limbajul de programare folosit predominant în acest proiect este *Python*, datorită suportului său extins pentru numeroase biblioteci și unelte care facilitează dezvoltarea ușoară și intuitivă a tehnologiilor bazate pe inteligență artificială, cum ar fi *PyTorch*. Python este apreciat pentru simplitatea și claritatea sa, aspecte esențiale în dezvoltarea rapidă și eficientă a unui astfel de proiect.

Pentru dezvoltarea nucleului asistentului personal Odin, a fost utilizat framework-ul *Rasa*. Rasa este un framework open-source destinat construirii de asistenți conversaționali și chatbots, oferind instrumente puternice pentru procesarea limbajului natural (NLP), antrenarea modelelor de învățare automată și gestionarea dialogului. Celelalte componente, care funcționează sub forma unor servicii distribuite, se bazează pe micro-framework-ul *Flask*. Flask este un micro-framework pentru web, scris în Python, care oferă flexibilitate și simplitate în crearea de aplicații web. Este ideal pentru proiecte mici și mijlocii, permițând dezvoltatorilor să extindă funcționalitățile aplicației în mod modular.

Modelele mari de limbaj (LLMs) au fost pregătite, încărcate și sunt utilizate cu ajutorul uneltelor oferite de bibliotecile *Transformers*, *Datasets* și *Accelerate*. Aceste biblioteci reprezintă unelte de nivel înalt care funcționează deasupra framework-ului *PyTorch*, oferind o interfață accesibilă pentru manipularea și antrenarea modelelor de învățare profundă.

5.1. Schema Generală a Aplicației

În capitolul anterior, s-a prezentat la un nivel înalt arhitectura conceptuală a sistemului, evidențiind componentele și legăturile dintre acestea. Acum vom coborî un nivel, prezentând schema generală a aplicației, o schemă complexă în care componentele sunt despachetate în module și se va evidenția comunicarea dintre acestea prin request-uri și răspunsurile modulelor la ele. În **Figura 5** este ilustrată schema generală a sistemului, în care se prezintă componentele principale împreună cu modulele lor cheie expuse, evidențiindu-se fluxul de procesare a comenzilor și modul în care modulele bot-ului interacționează între ele pentru a ajunge de la o comandă în

text la un răspuns vocal potrivit, cu sens, intonație și cursivitate. După aprofundarea la nivel înalt a acestei scheme, vom începe să disecăm fiecare componentă în parte, dar momentan este importantă urmărirea fluxului de informație de-a lungul pipeline-ului de procesare al asistentului.

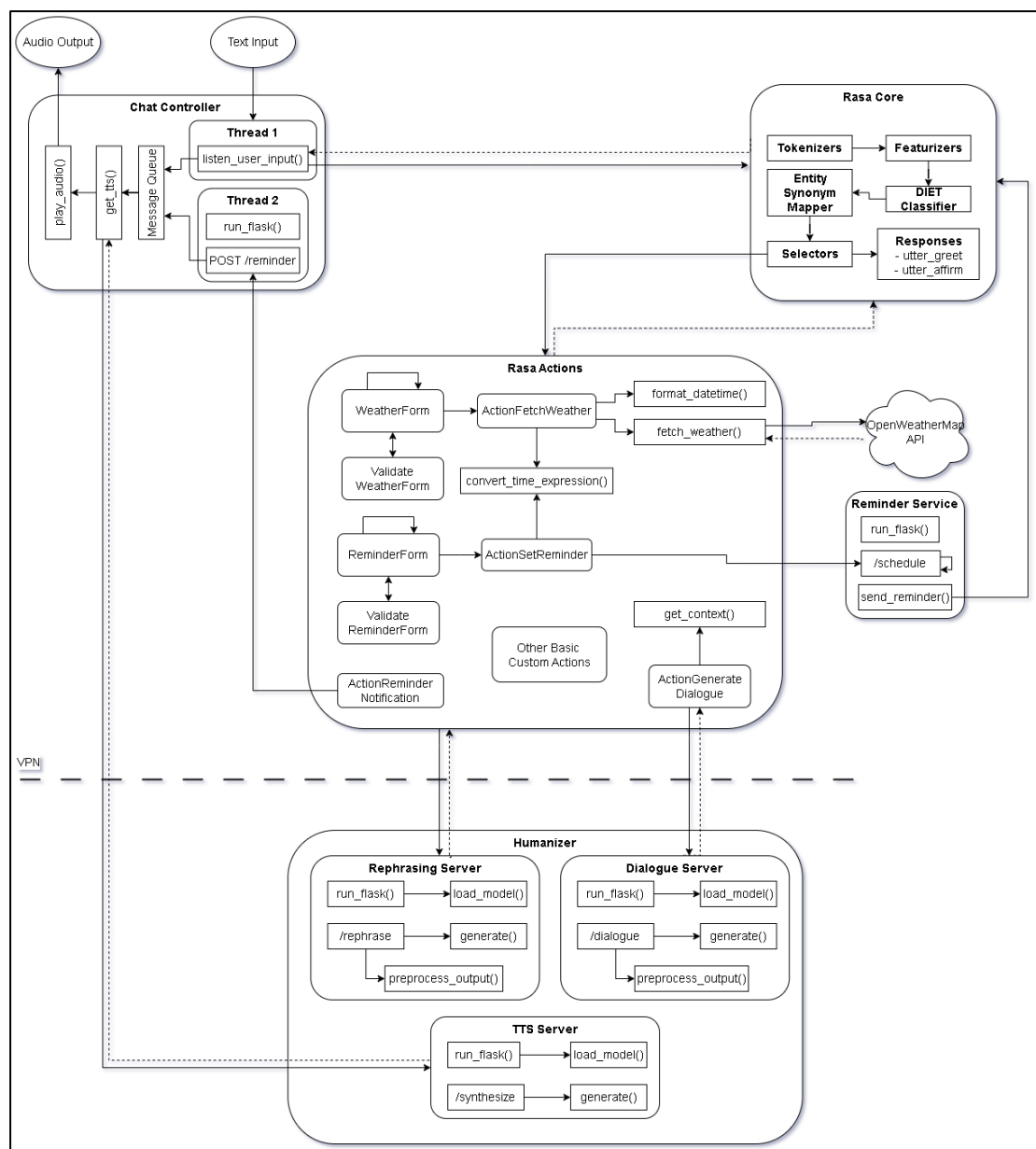


Figura 5: Schema Generală a Sistemului

5.1.1. Fluxul de Procesare a Comenzilor

După cum este evidențiat în schema insert figura, asistentul primește ca intrare un șir de caractere introdus de utilizator de la tastatură, care este așteptat și interceptat în metoda *listen_user_input()*. Se poate observa prezența a două thread-uri în componenta de control al mesajelor (*Chat Controller*). Acest fapt se datorează naturii funcțiilor de preluare a intrării de la utilizator, precum cea folosită în proiect *input('You: ')*, care evidențiază în consolă rândul utilizatorului de a vorbi, urmată de funcția *print(f'Bot: {message}')*, care afișează rândul și răspunsul asistentului. Revenind,

având un thread blocat care așteaptă comanda de la utilizator, componenta de control al mesajelor are nevoie de încă un fir de lucru pentru rularea serviciului care așteaptă în mod asincron remindere. Acest al doilea thread permite bot-ului să gestioneze și să răspundă la remindere fără a întrerupe fluxul principal de dialog al utilizatorului.

După introducerea comenzii, aceasta este trimisă la serverul local care rulează nucleul Rasa la adresa *localhost:5005/webhooks/rest/webhook*, declanșând pipeline-ul de înțelegere a limbajului natural (Natural Language Understanding, NLU) al nucleului, discutat la nivel înalt în capitolul 4, evidențiat în **Figura 2**. La finalul pipeline-ului NLU, se decide de către componentele Selectors ce acțiune se va executa: fie folosirea unui răspuns simplu, predefinit, fie apelarea unei acțiuni personalizate din serverul de acțiuni, care rulează separat la adresa *localhost:5055*, pentru a încuraja decuplarea și testarea mai ușoară. La alegerea unei acțiuni din ecosistem, firul de execuție se ramifică foarte mult în următoarele ramuri principale:

- Acțiunea *ActionFetchWeather*, fiind cea mai complexa dintre toate, dispune de cele mai multe module întrebuințate. Aceasta are nevoie de validarea formularului asociat, specificarea slot-ului *city* și a minim unuia dintre slot-urile *offset_time* sau *specific_time*. Urmează agregarea convertirea valorilor de timp din limbaj natural într-o data concreta cu tipul *datetime* specific Python. În urma convertirii, acțiunea apelează o funcție ce funcționează ca un wrapper peste API-ul OpenWeatherMap, pentru a găsi prognoza disponibilă pentru cea mai apropiată data de data specificată de utilizator. Într-un final, este necesară formularea unui răspuns concret, în care sunt agregate toate informațiile despre vreme primite, precum temperatura, condiția meteo și viteza vântului, împreună cu data și ora exactă într-un format elegant.
- Acțiunea *ActionSetReminder* are la început un comportament similar acțiunii *ActionFetchWeather*, în ideea necesității validării slot-urilor de *task* și *offset_time* sau *specific_time*. De asemenea conversia de timp pentru data exactă a reamintirii este identică. Mai departe se apelează serviciul de reamintiri (*Reminder Service*) pentru planificarea în viitor a reamintirii, în funcție de timpul specificat de utilizator.
- Acțiunea *ActionReminderNotification* are rolul de a efectua reamintirea planificată în acțiunea descrisă precedent. Datorită naturii asincrone a reminderelor și a request-urilor HTTP de tip POST, această notificare nu se poate face prin nucleul Rasa, înapoi la *Chat Controller*, și este nevoie de apelarea endpoint-ului dedicat din controlorul de mesaje pentru trimiterea mesajului de tip reminder.
- Acțiunea *ActionGenerateDialogue* servește atât ca acțiune desemnată pentru facilitarea conversației dintre utilizator și bot, cât și pasul executat în cazul în care botul nu știe ce să facă, și declanșează intenția *nlu_fallback*. În primul rând, trebuie reconstituirea contextului de mesaje, prin agregarea ultimelor 5 mesaje din dialogul asistent-utilizator și separarea acestora prin '*n*', trimiterea unui

POST request la *192.168.0.107:5002/dialogue*, adresa la care rulează serviciul de conversație care are la baza un LLM dedicat.

- Alte acțiuni Rasa sunt acțiuni care nu interacționează cu exteriorul și manipulează doar sloturile și entitățile din ecosistemul bot-ului, precum acțiunile care interacționează cu lista de cumpărături *ActionAddShoppingList*, *ActionReadShoppingList*, *ActionClearShoppingList*; acțiunea simplă de spus glume *ActionTellJoke*; acțiunile de întâmpinare *ActionHandleGreeting* și *ActionHandleGoodMorning*; acțiunile de administrare a stării de spirit a asistentului *ActionHandleGoodIntent*, *ActionHandleBadIntent* și *ActionHandleApology*; iar la final acțiunile de pornire/oprire a bot-ului *ActionHandleWakeUp* și *ActionHandleShutDown*.

Este de notat structura similară a majorității acțiunilor existente: există un set de răspunsuri predefinite (între 2 sau 3) dintre care se alege unul la întâmplare cu ajutorul librării funcției *random.choice(responses)* din Python. Acest răspuns este apoi trimis la serviciul de reformulare, la adresa *192.168.0.107:5001/rephrase* împreună cu valoarea extrasă din slot-ul de mood al asistentului, pentru reformularea răspunsului, în așa fel încât să se elimine natura repetitivă ale răspunsurilor chatbot-ului la comenzi.

Mai departe, după executarea unei acțiuni, nucleul Rasa va răspunde la POST request-ul făcut inițial din Chat Controller cu răspunsul în text la comanda utilizatorului, urmând ca acesta să fie trimis ca parametru la funcția *get_tts()* care apelează endpoint-ul serviciului de umanizare *192.168.0.107:5003/synthesize* cu un POST request, răspunsul urmând să fie redat pe difuzor ajutorul funcției *play_audio()*.

5.2. Controlorul de Mesaje

În primul rând, ca intrare în procesul de prelucrare a sistemului, avem o comandă în text, introdusă de la tastatură în consola specifică procesului asociat script-ului *chat_controller.py*. Întregul proces de captare a comenzii de la tastatură, trimiterea acesteia mai departe și apelarea funcțiilor pentru redarea audio este gestionat de funcția *listen_user_input()*, care rulează într-o buclă infinită pe un thread special instanțiat și începe cu apelul funcției *input('You: ')*, blocând firul de lucru curent în așteptarea comenzii utilizatorului. După primirea comenzii, se trimite un POST request la nucleul Rasa cu textul introdus, către adresa implicită pe care rulează serverul nucleului Rasa, *localhost:5005/webhooks/rest/webhook*, pentru a începe procesarea comenzii. Detaliile privind implementarea nucleului asistentului vor fi prezentate în sub-capitolul viitor, dar momentan vom reveni la momentul în care modulul de control al mesajelor primește răspunsul la request-ul menționat anterior și îl adaugă în coada de mesaje.

Este esențială instanțierea unui al doilea thread în script-ul de control al conversației pentru primirea mesajelor de reamintire (reminders) într-o manieră asincronă. Cel de-al doilea fir de lucru rulează un serviciu Flask pentru expunerea unui

endpoint la adresa *localhost:5001/reminder*, care interceptează remindere trimise din nucleul Rasa la un timp determinat de utilizator după comanda de setare. Funcția de interceptare a reamintirilor adaugă mesajele primite în coada specială de mesaje.

Coadă de mesaje este o componentă crucială pentru sincronizarea primirii răspunsurilor de la asistent în cele două thread-uri menționate mai sus. Având această abordare, ne asigurăm că mesajele nu se vor pierde în cazul în care controlorul mesajelor primește un răspuns la o comandă și un reminder în același timp, eliminând astfel posibile erori din program. Această coadă dedicată rulează în thread-ul principal într-o buclă infinită, care este blocată de funcția *message_queue.get()* în așteptarea primirii unui răspuns, fie din firul conversației cu utilizatorul, fie din firul specific reminderelor.

După primirea unui mesaj, intervine componenta de umanizare implementată pe hardware dedicat la distanță, modulul text-vorbire (eg. Text-to-Speech, TTS), care expune endpoint-ul *192.168.0.107:5003/synthesize*, apelat în funcția *get_tts(text)*. Răspunsul primit sub format *.wav* este verificat cu ajutorul condiției *if audio := get_tts(message):* și redat prin funcția *play_audio(audio)*, care utilizează librării externe specializate pentru interacțiunea cu dispozitivele de ieșire audio ale Windows-ului.

5.3. Nucleul Rasa

Următoarea verigă, posibil și cea mai importantă, din lanțul de procesare a comenzilor este nucleul asistentului, implementat pe baza framework-ului din Python numit Rasa. Acest framework este specializat și echipat cu unelte pentru dezvoltarea roboților de conversație (chatbots). Rasa oferă utilizatorului tehnici și algoritmi performanți și optimizați pentru rezolvarea și abstractizarea unor provocări care, în mod normal, ar necesita cunoștințe detaliate despre domeniile respective. Datele de intrare, sub formă de text, trec printr-o serie de componente de procesare, fiecare având un rol diferit, dar esențial pentru rezultatul final.

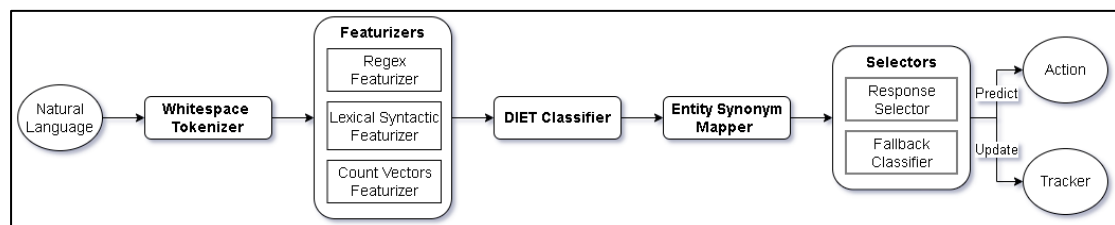


Figura 6: Pipeline-ul Rasa NLU

5.3.1. Tokenizarea

Primul pas în fluxul de procesare a limbajului natural (NLP) este tokenizarea, un proces care presupune divizarea textului în unități mai mici, numite tokeni. În contextul tehnicii de tokenizare folosite de Rasa, Whitespace Tokenizer, se presupune că textul este limbaj natural cursiv și inteligibil, iar împărțirea tokenilor se face pe baza

spațiilor dintre cuvinte. De asemenea, semnele de punctuație sunt păstrate ca tokeni separați, deoarece conțin informații esențiale pentru analiza sintactică și semantică ulterioară. Un exemplu concret al acestui proces ar fi extragerea din mesajul „Hey, Odin, how are you?” a tokenilor {‘Hey’, ‘,’, ‘Odin’, ‘,’, ‘how’, ‘are’, ‘you’, ‘?’}.

5.3.2. Caracterizarea

În urma tokenizării, a doua etapă este "caracterizarea" (featurizing) textului, un proces prin care textul brut este transformat în caracteristici (features) numerice sau categorice care sunt utilizate de algoritmi de învățare automată (machine learning). Acest pas este crucial pentru modelele ML, care nu pot lucra direct cu date textuale și au nevoie de conversia acestora în reprezentări numerice pentru a putea învăța și a face predicții. Etapa de featurizing are trei sub-pasi care trebuie executați pentru a extrage toate caracteristicile utile, fiecare reprezentând un tip diferit de featurizer:

Regex Featurizer creează o reprezentare vectorială a mesajului utilizatorului utilizând expresii regulate (regex). În timpul antrenării, *RegexFeaturizer* creează o listă de expresii regulate definite în datele de antrenare. Pentru fiecare regex, se va seta o caracteristică ce indică dacă această expresie a fost găsită în mesajul utilizatorului sau nu. Toate aceste caracteristici vor fi ulterior utilizate de un clasificator de intenții sau un extractor de entități. Un exemplu ar fi expresia regulată `[0-9]{2}:[0-9]{2}` pentru a detecta orele, iar în propoziția „I want to book a table at 19:00”, va detecta că expresia 19:00 corespunde regex-ului menționat.

Count Vectors Featurizer are rolul de a crea o reprezentare „sac de cuvinte” (bag-of-words) a textului primit. Utilizând *CountVectorizer* din *sklearn*, transformă mesajul utilizatorului, intenția și răspunsul în reprezentarea bag-of-words. De exemplu, pentru propoziția „The fox found the dog”, am avea vocabularul `[the, fox, found, dog]` și matricea de contorizare `[2, 1, 1, 1]`.

Lexical Syntactic Featurizer creează caracteristici lexicale și sintactice pentru un mesaj al utilizatorului, folosind o fereastră glisantă pentru a parcurge fiecare token din mesaj conform caracteristicii configurației.

5.3.3. Clasificarea Intențiilor și Extragerea Entităților

Următoarea componentă de procesare din lanțul nucleului Rasa este clasificatorul DIET (Dual Intent and Entity Transformer), care reprezintă o arhitectură multi-task pentru clasificarea intențiilor și recunoașterea entităților. Utilizează un transformer partajat pentru ambele sarcini, permițând modelului să învețe simultan să clasifice intențiile și să extragă entități.

Clasificarea intențiilor înseamnă identificarea scopului sau intenției din spatele unui mesaj al utilizatorului, dintr-un set bine definit de intenții. De exemplu, când cineva spune „I want to book a table”, intenția ar putea fi `book_table`. Transformerul

din DIETClassifier ia propoziția sub forma de features generate de pasul anterior și o transformă într-o reprezentare numerică (vector). Fiecare intenție cunoscută are, de asemenea, o reprezentare vectorială în același spațiu. Transformerul compară vectorul propoziției utilizatorului cu vectorii intențiilor cunoscute și calculează cât de asemănătoare sunt acestea. Aceasta se face folosind o metodă matematică numită *dot-product loss*, care se bazează pe compararea produselor scalare dintre vectorul propoziției și vectorii corespunzători intențiilor. Un produs scalar mare indică o similaritate mare între vectori, sugerând că intenția respectivă este probabil cea corectă pentru propoziția dată. Modelul returnează intenția prezisă cu un grad de încredere mai mic sau egal cu 1 (de exemplu, 0.78) și o listă de alte intenții posibile ordonate după încredere, toate intențiile din listă având suma gradelor de încredere egală cu 1.

Extragerea entităților înseamnă identificarea și extragerea unor părți specifice din mesaj care au o semnificație specială, cum ar fi numere, date, locații, etc. De exemplu, în propoziția „*I want to book a table at 19:00*”, entitatea ar putea fi 19:00. Spre deosebire de pasul anterior al vectorizării unei întregi propoziții, acum transformerul numeric va produce câte un vector numeric pentru fiecare cuvânt din propoziție. După ce fiecare cuvânt este transformat într-un vector, aceștia sunt trimiși printr-un strat CRF (Conditional Random Field). CRF-ul este un algoritm de etichetare care decide dacă fiecare cuvânt sau grup de cuvinte reprezintă o entitate specifică. Algoritmul ia în considerare vecinătatea fiecărui cuvânt în contextul său și faptul că un cuvânt poate să fie doar parte a unei entități. În final, modelul returnează lista de entități identificate, împreună cu pozițiile lor de început și sfârșit în text și gradul de încredere asociat fiecărei entități.

Antrenarea acestui modul depinde foarte mult de datele furnizate de utilizator în fișierul *nlu.yml*, desemnat special pentru specificarea denumirilor intențiilor împreună cu un set expresiv de exemple în limbaj natural, în care se pot include și entitățile asociate intențiilor, pentru antrenarea clasificatorului DIET. De exemplu, dacă dorim ca modelul să învețe intenția *book_table* și să extragă ora pentru rezervare, vom furniza în fișierul *nlu.yml* exemple precum „*I want to book a table at [19:00](time)*” sau „*May I please book a table for me and my girlfriend at [21:00](time) at your restaurant?*” Această posibilitate de a crea un set de date de antrenament printr-un număr relativ redus de exemple, cu entități specificate direct din text, cu valoarea între paranteze pătrate și numele entității între paranteze rotunde, face clasificatorul DIET să fie o unealtă foarte puternică, folositoare și prietenoasă cu dezvoltatorul. Astfel, procesul de implementare a unui chatbot devine mult mai eficient și accesibil.

5.3.4. Maparea Sinonimelor

După extragerea entităților, urmează un pas de normalizare a acestora în Entity Synonym Mapper, o componentă din pipeline-ul Rasa care ajută la uniformizarea modului în care entitățile sunt reprezentate. Mai exact, asigură că diferite variante ale aceleiași entități sunt mapate la o valoare unică și consistentă. De exemplu, am putea

avea două mesaje de genul „*I will be arriving in Cluj in 30 minutes*” și „*I will be arriving in CJ in 30 minutes*” și entitatea sub numele de *city*. Conform pasului anterior, clasificatorul DIET ar extrage entitățile ca fiind ‘*Cluj*’ și ‘*CJ*’, moment în care sistemul ar putea trata ‘*Cluj*’ și ‘*CJ*’ ca entități diferite, chiar dacă se referă la același loc. Aceasta poate cauza inconsistențe și confuzii în sistemul de chatbot. Remediul pentru această problemă este componenta Entity Synonym Mapper, care se asigură că entitățile sinonime sunt mapate la aceeași valoare. Acest pas de normalizare garantează coerența și consistența datelor, permițând sistemului să interpreteze corect și unitar informațiile furnizate de utilizator.

5.3.5. Selectarea Răspunsului

Ca ultim pas de procesare a comenzii din pipeline-ul Rasa, selectorii sunt responsabili pentru alegerea răspunsului potrivit al chatbot-ului pe baza intrării utilizatorului și a intențiilor clasificate anterior.

Fallback Classifier are rolul de a clasifica un mesaj cu intenția *nlu_fallback* dacă scorurile de clasificare a intențiilor sunt ambigue. În acest caz, încrederea este setată la valoarea pragului de fallback. Acesta intervine atunci când clasificatorul anterior de intenții nu poate clasifica o intenție cu un scor de încredere mai mare sau egal cu pragul setat, sau când scorurile de încredere ale celor mai bine două clasificate intenții sunt prea apropiate (mai mici decât *ambiguity_threshold*). Rasa permite configurarea explicită a unui răspuns în cazul declanșării intenției de *nlu_fallback* în fișierul de reguli *rules.yml*.

Response Selector este folosit pentru a construi un model de regăsire a răspunsurilor, care prezice direct răspunsul bot-ului dintr-un set de răspunsuri candidate. Are o arhitectură similară cu cea a clasificatorului DIET, vectorizează intrările utilizatorului și etichetele răspunsurilor în același spațiu și urmează aceeași arhitectură neurală și optimizare.

5.3.6. Structura Nucleului Rasa

Pentru a înțelege mai bine modul de procesare a comenzilor de către un chatbot realizat cu Rasa, este importantă înțelegerea structurii proiectului în care sunt definite toate elementele necesare în compoziția unui astfel de asistent personal și legăturile dintre acestea, de la intenții și date de antrenament, la acțiuni și răspunsuri, de entități, sloturi și formulare, până la logica de business externă.

Directorul „actions” cuprinde tot ce înseamnă logică și cod arbitrar executat de către chatbot. Definește atât acțiunile simple și complexe, cât și module externe folosite în procesarea și preluarea datelor de la API-uri din exterior.

- *actions.py* – Fișierul Python care conține definirea tuturor acțiunilor implementate de Odin. Mai multe detalii despre tipurile de acțiuni vor fi discutate în secțiunea următoare.
- *nl_time_converter.py* – Un modul care conține funcții importante de conversie a timpului specificat în limbaj natural într-o ștampilă de timp concretă (tipul de date *datetime*), folosită ca parametru la preluarea prognozei meteo corespunzătoare unei anumite date din viitor. Modulul dispune și de o funcție de conversie a ștampei de timp din memorie înapoi în limbaj natural elegant (ex. „10-02-2024” convertit în „10th of February 2024”).
- *weather_api.py* – Funcții pentru preluarea informațiilor legate de vreme de la *OpenWeatherMap*, care oferă un API accesibil și gratuit pentru primele cereri. Modulul dispune de două funcții: una pentru vremea din prezent și alta pentru prognoza meteo. Aceste funcții învelesc HTTP request-ul către *OpenWeatherMap*, ocupându-se atât de pregătirea parametrilor necesari în încărcătura (cheia API, orașul și data curentă), cât și de tratarea cazurilor de eroare. Informațiile extrase de către aceste două funcții sunt trimise mai departe la acțiunea personalizată *action_fetch_weather*.

Directorul „*data*” cuprinde toate fișierele necesare pentru pregătirea datelor de antrenament și specificarea comportamentului dorit al deciziilor chatbot-ului.

- *nlu.yml* – Fișierul care cuprinde toate denumirile intențiilor împreună cu datele de antrenament corespunzătoare lor, sub forma „*-intent: {nume} | examples: {examples}*”. Definirea exemplilor permite și specificarea entităților asociate unei intenții, având partea din exemplu ce simbolizează entitatea între paranteze pătrate, urmată de numele entității între paranteze rotunde (ex. „*Add milk to the shopping list*”).
- *chatito* – Un sub-director în care se păstrează toate fișierele cu extensia *.chatito*, care poartă numele unui instrument open-source care ajută la generarea automată de date de antrenament pentru chatboți. Folosind un limbaj simplu, Chatito permite definirea intențiilor și entităților unui chatbot și crearea rapidă a multiplelor variații ale exemplelor de antrenament pe baza unor șabloane de bază și a unor cuvinte sau grupuri de cuvinte de substituție în șabloanele definite. Un exemplu intuitiv ar fi intenția 'add_shopping_list' pentru care avem șablonul „*Add ~[item] to the ~[list]*”, și substituțiile 'milk', 'bread', 'butter' pentru [item], iar substituțiile 'shopping list', 'basket' pentru [list].
- *data_aggregation.py* – Un script care nu corespunde structurii standard a unui proiect Rasa, implementat pentru ușurința agregării datelor de antrenament în constantă schimbare în fișierul *nlu.yml*. Scriptul se folosește de toate fișierele cu extensia *.chatito* precizate mai sus prin invocarea comenzii „*chatito {file_name} -format=rasa*”, urmată de scrierea exemplilor furnizate de fiecare

comandă în fișierul *nlu.yml*, respectând formatul acestuia pentru a evita conflictele validării datelor de antrenament. Având acest script se elimină interacțiunea manuală cu fișierul datelor de antrenament, putând acum cu ușurință să fie adăugate, manipulate sau șterse doar din fișierele *.chatito* și rularea scriptului.

- *rules.yml* – Acest fișier facilitează definirea comportamentului și a deciziilor bot-ului la detectarea anumitor intenții. Se specifică intenția și acțiunea ce se va declanșa în urma intenției respective. În acest fișier se declară deschiderea buclelor de validare a formularelor, dar mai multe detalii despre aceste tipuri de acțiuni vor fi prezentate în secțiunea următoare. Formatul standard este „*-rule: {name} | steps: -intent: {intent} | -action: {action}*”.
- *stories.yml* – Un tip de date de antrenament folosite pentru a antrena modelul pe baza unor fire conversaționale predefinite. Ajută bot-ul să generalizeze mai bine căi conversaționale nemaiîntâlnite. În cazul proiectului nostru, fișierul este gol deoarece sunt mult prea multe intenții și moduri în care conversația dintre Odin și utilizator poate decurge, astfel definirea poveștilor în fișier ar fi inutilă.

Directorul „*models*” stochează modelele antrenate cu comanda „*rasa train*”. Modelele sunt stocate sub denumiri utile și intuitive, cu formatul „*{data}-{ora}-{denumire_aleatorie}.tar.gz*”, spre exemplu, Odin trăiește de fapt sub denumirea „*20240626-124316-tall-goldbricker.tar.gz*”. Acest mod de stocare a fișierelor ajută comanda de încărcare a modelului în memorie să selecteze cea mai recentă iterație.

Directorul „*config.yml*” este folosit pentru specificarea componentelor Rasa care funcționează „under the hood”, precum întregul pipeline discutat anterior și politicile de predicție ale acțiunii. În cazul nostru, pipeline-ul este neschimbat, cel default cu care vine Rasa „out of the box”, deoarece este unul bine definit, cu performanțe ridicate în procesarea limbajului natural al utilizatorului. În ceea ce privește politicile, s-au făcut mici modificări pentru a pune în valoare intenția prezisă în momentul în care nicio altă intenție nu e potrivită, anume *nlu_fallback*. În loc să fie un asistent personal standard care cere reîncercarea comenzii de către utilizator în cazul neînțelegerii, Odin folosește *nlu_fallback* pentru a genera dialog, astfel menținând utilizatorul constant captivat în discuție.

Directorul „*domain.yml*” deține toate variabilele prezente în ecosistemul bot-ului, având un nume foarte sugestiv. Acesta este un mod centralizat de păstrare a informațiilor din tot domeniul asistentului. Stochează numele intențiilor și al acțiunilor, definirea sloturilor (tip de date, mod de validare, valori inițiale), formularele (sloturile necesare pentru validare), numele tipurilor de entități și răspunsurile simple în text. Acest fișier este esențial pentru configurarea și funcționarea corectă a bot-ului, deoarece toate componentele principale ale ecosistemului asistentului sunt definite și organizate aici.

5.3.7. Acțiunile Rasa

În contextul Rasa, orice reacție a bot-ului la o comandă se numește o acțiune, iar acestea pot fi de mai multe feluri, de la un simplu răspuns în text, până la o funcție

complexă care rulează orice fel de cod definit de către dezvoltator (apeluri API, interogări la baza de date desemnată pentru Rasa, logică complexă etc.). Ecosistemul acțiunilor din Rasa se dovedește a fi extrem de folositor și flexibil în implementarea răspunsurilor calitative la comenzi. Fiecare acțiune personalizată moștenește clasa părinte de *CustomAction*, care oferă acces la întregul mediu al nucleului Rasa, dând astfel posibilitatea dezvoltatorului să citească și să manipuleze entitățile, sloturile și răspunsurile din acest mediu al chatbot-ului.

În dezvoltarea asistentului Odin, un chatbot relativ complex, s-au folosit toate acțiunile puse la dispoziție de către Rasa, pentru a profita la maxim de posibilitățile de dezvoltare a unui chatbot cât mai calitativ. Mai departe, vom detalia fiecare acțiune în parte, punând în evidență întrebuințarea fiecăreia în răspunsurile asistentului personal la comenzile utilizatorului.

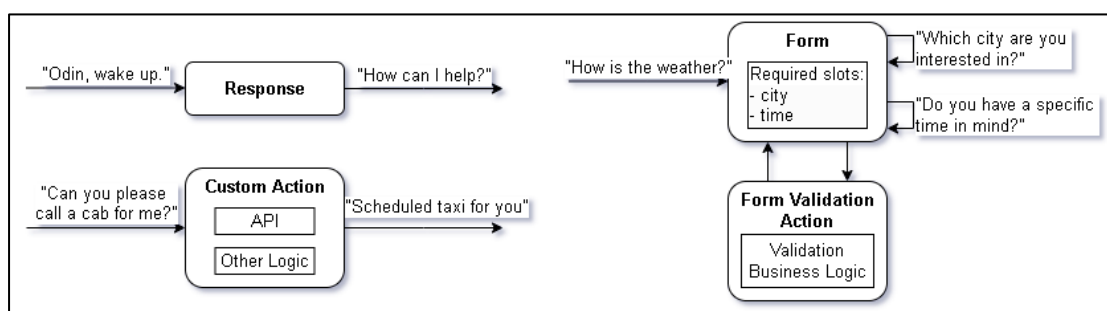


Figura 7: Tipurile de Acțiuni Rasa

Răspunsurile reprezintă reacții simple în text ale bot-ului și sunt acțiuni care nu trebuie definite explicit într-o clasă în fișierul *actions.py*. Este suficient să fie denumite în fișierul *domain.yml*, împreună cu asocierea lor cu anumite intenții în fișierul *rules.yml*. Astfel, bot-ul face legătura între intențiile detectate și răspunsul asociat lor. În contextul proiectului, acesta este cel mai puțin utilizat tip de acțiune, folosit doar în cadrul validării formularelor, deoarece majoritatea răspunsurilor pot fi o selecție dintr-un set mai mare de răspunsuri, urmând ca acestea să fie reformulate pe baza stării de spirit a asistentului. Acest fapt introduce necesitatea unui apel API la componenta de umanizare, dar mai multe detalii despre acest aspect vor fi discutate mai târziu în lucrare. Un exemplu de utilizare al răspunsurilor ar fi validarea formularului de vreme, în care utilizatorul poate întreba „*How is the weather outside?*”, iar Odin să folosească răspunsul *utter_ask_city* pentru a solicita confirmarea slotului absent *city*. Prin urmare, asistentul ar răspunde cu „*Which city are you interested in?*” într-o buclă, până la umplerea slotului cu entitatea corespunzătoare și validarea formularului de vreme.

Acțiunile Personalizate (Custom Actions) sunt acțiuni definite explicit în fișierul *actions.py* și permit rularea oricărui cod arbitrar. Acest tip de acțiuni este predominant în dezvoltarea lui Odin, deoarece acesta utilizează componente care rulează în afara ecosistemului Rasa, fiind necesară facilitarea comunicării între nucleu și exterior prin definirea unor metode consistente de comunicare. Acțiunile personalizate au o structură bine definită și trebuie să moștenească clasa *Action* expusă de Rasa, să implementeze metoda *name()*, care va returna numele sau identificatorul

acțiunii regăsit în fișierul de domeniu *domain.yml*, precum și să implementeze metoda *run()*, în care se adaugă codul arbitrar dorit și se poate accesa și manipula ecosistemul nucleului împreună cu toate variabilele, entitățile și sloturile din acesta. În contextul proiectului, acțiunile personalizate au o structură comună, care constă în execuția unei logici specifice acțiunii, selectarea unui răspuns dintr-un set de răspunsuri predefinite și apelul spre componenta de umanizare (reformulare) a răspunsului selectat pe baza stării de spirit a asistentului.

Cel mai complex tip de acțiune personalizată implică trei entități posibile: *city* (orașul pentru care se va găsi prognoza meteo), *specific_time* (entitate ce reprezintă o referință specifică de timp, de exemplu: *tonight*, *now* sau *at 3PM*) și *offset_time* (entitate ce se referă la un deplasament de timp față de timpul actual, de exemplu *in 3 hours*, *next week*, *in two days*). După umplerea sloturilor, acțiunea apelează un modul extern folosit pentru transformarea entităților din sloturile referitoare la timp în limbaj natural într-un deplasament de timp concret și un format acceptat de către API-ul pentru prognoze meteo. O altă funcție externă este apelată, având ca argumente orașul și data exactă, pentru preluarea prognozei meteo de pe *OpenWeatherMap*, o organizație ce oferă acces ușor și gratuit la prognoze meteo precise pe baza unui API. Sunt alese doar datele relevante din prognoza primită și sunt agregate într-un răspuns în limbaj natural, împreună cu răspunsul unei alte funcții externe ce are rolul de a formata data din tipul *datetime* într-un șir de caractere elegant (ex., „01-02-2001” convertit în „*First of February, 2001*”). Ca ultim pas în această acțiune ar fi cel comun tuturor, acela de reformulare.

Formularele sunt un tip special de acțiuni, concepute pentru a gestiona logica mai complexă, în care colectarea tuturor informațiilor necesare pentru răspuns ar necesita mai multe iterații de dialog. Acestea sunt definite în fișierul *rules.yml* doar prin specificarea denumirii lor și a sloturilor implicate în formular. Chatbot-ul poate fi programat să deschidă un formular la detectarea unei anumite intenții și să ceară structurat de la utilizator validarea fiecărui slot care nu a fost specificat. Un exemplu concret ar putea fi declanșarea formularului la detectarea intenției de *fetch_weather*. Formularul va rula într-o buclă și va întreba structurat utilizatorul despre fiecare slot: când utilizatorul zice „*Can you fetch the weather for me?*”, formularul *weather_form* declanșat va determina bot-ul să răspundă cu „*Which city are you interested in?*”. După specificarea unui oraș, asistentul va răspunde cu „*Do you have a specific time in mind?*”, iar în cele din urmă utilizatorul va specifica și timpul, moment în care toate sloturile ar fi validate și formularul se poate închide, declanșând acțiunea *action_fetch_weather*.

Acțiunea de Validare a Sloturilor este un ultim tip special de acțiune care gestionează extragerea și validarea sloturilor. Această acțiune poate facilita validarea unui slot în cazuri speciale fără interacțiune din partea utilizatorului sau invalidarea acestuia în cazul în care datele specificate de utilizator sunt invalide. În cazul asistentului nostru personal, acțiunea de validare intervine în *weather_form* la specificarea timpului. Fiind prezente două sloturi pentru timp (*specific_time* și

offset_time), este necesară umplerea doar a unuia dintre ele pentru validarea formularului. Prin urmare, această acțiune este apelată mereu în bucla *weather_form* și verifică umplerea a minimum unuia din sloturi, moment în care elimină necesitatea validării pentru cel de-al doilea.

5.4. Serviciul de Reamintiri (Reminders)

Un reminder înseamnă un mesaj trimis de bot cu scopul de a aduce aminte utilizatorului de un anumit eveniment, sarcină sau obligație la un moment bine definit din viitor. Data fiind această definiție, se observă natura asincronă a unui reminder, fapt care face implementarea unui serviciu de remindere intern în nucleul bot-ului dificilă și de mare complexitate. Declanșarea unui mesaj de reminder se face la un timp specificat de utilizator (ex. 10 minute sau 10 ore), ceea ce ne lasă cu câteva variante: folosirea unui temporizator și inițializarea acestuia cu numărul de secunde corespunzător, blocarea procesului pentru același număr de secunde sau o buclă care compară data curentă *datetime.now()* cu data din viitor extrasă din deplasamentul de timp specificat de utilizator. Toate aceste variante ar introduce complexitate inutilă în procesul ce rulează nucleul lui Odin și ar putea crește predispoziția la erori.

Soluția propusă ar fi decuplarea acestei logici față de nucleu și implementarea unui proces separat care să ruleze un server, comunicând cu nucleul prin HTTP. Această abordare nu ar necesita modificări la comportamentul nucleului Rasa, doar definirea a două intenții noi, *reminder* și *set_reminder*, cu acțiunile corespunzătoare.

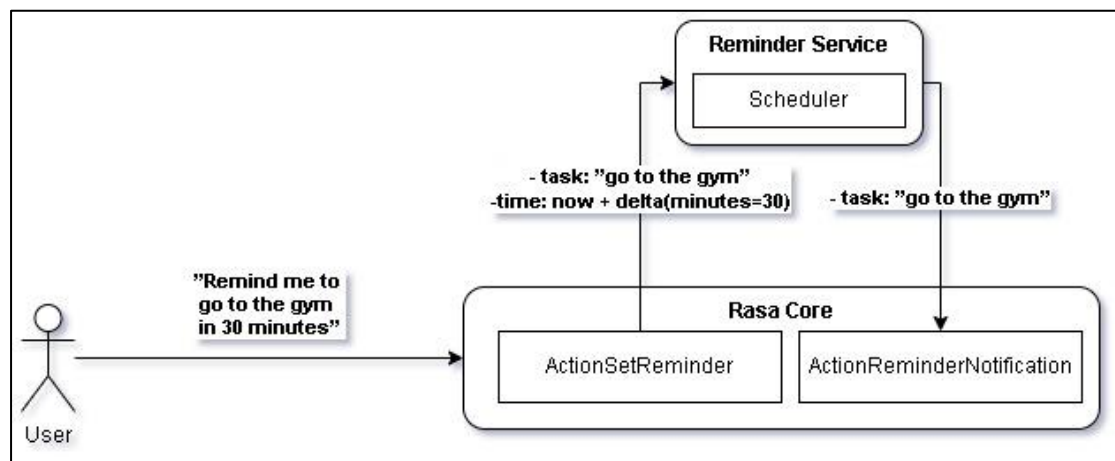


Figura 8: Sistemul de Reamintiri

5.4.1. Intenția 'set_reminder'

Intenția *set_reminder* va conține slotul esențial numit *task*, care reprezintă sarcina propriu-zisă ce trebuie amintită. Similar cu intenția de *fetch_weather*, aceasta va conține și cele două sloturi *specific_time* și *offset_time*, fiind necesară validarea a

minim unuia dintre cele două pentru executarea acțiunii. Aceasta are rolul de a facilita programarea reamintirii viitoare cu un HTTP request către serviciul de remindere, care are ca parametri sarcina (slot-ul task) și deplasamentul de timp în secunde de la momentul actual când se va executa reamintirea. Acțiunea constă în extragerea unei date exacte în tipul *datetime* din Python, corespunzătoare deplasamentului de timp specificat în limbaj natural. Pentru aceasta, vom folosi același convertor utilizat pentru intenția *fetch_weather* la extragerea timpului, implementat în *nl_time_converter.py*. Cu ajutorul datei exacte extrase, vom calcula diferența de secunde dintre timpul specificat de utilizator și momentul din prezent, valoare pe care o vom folosi la programarea reamintirii în serverul de remindere. Mai multe detalii despre această funcționalitate vor fi prezentate în următoarea sub-secțiune.

5.4.2. Serverul de Remindere

Serviciul propriu-zis de reamintiri va rula pe un server de Flask în Python, un framework utilizat pentru dezvoltarea aplicațiilor web. Flask va fi folosit pentru orice microserviciu implementat în proiect, fiind ușor și flexibil, ideal pentru servicii de dimensiuni mici, precum cele din proiect. Vom profita de funcționalitatea Flask de planificare (scheduling) și vom crea un endpoint la ruta */schedule*. Acesta va primi parametri necesari (task și număr de secunde) și va programa execuția la timpul potrivit unei funcții care va face un POST request înapoi la nucleu, conținând un mesaj cu format specific care encapsulează task-ul de amintit, astfel încât nucleul să detecteze intenția *reminder_notification*.

5.4.3. Intenția ‘reminder_notification’

Această intenție este definită cu exemple într-un format specific, identic cu cel al mesajului trimis de către serverul discutat anterior, nu în limbaj natural (ex. „*reminder_notification: [{task_example}](task)*”), și are rolul de a declanșa acțiunea specifică de notificare a utilizatorului la momentul potrivit. Acțiunea extrage din nou task-ul și preprocesează textul înainte să îl trimită mai departe. De exemplu, la setarea unui reminder utilizatorul va zice „*Odin, please remind me to wash my clothes in 30 minutes.*”, bot-ul va extrage task-ul „*to wash my clothes*”, dar când va reaminti peste timpul specificat va trebui să spună „*Paul, you have asked me to remind you to wash your clothes*”. Prin urmare, preprocesarea constă în schimbarea pronumelor de la persoana I la persoana a II-a. În continuare, asistentul va trimite mesajul cu reamintirea la controlorul de mesaje, de asemenea printr-un POST request, iar acesta îl va adăuga în coada de mesaje, urmând ca mesajul să fie rostit.

5.4.4. Argumentarea Decuplării Serviciului

Cum am spus, decuplarea serviciului de remindere de nucleu a fost aleasă datorită naturii asincrone a reamintirilor. Implementarea unui serviciu de remindere intern, direct în nucleul bot-ului, ar fi fost complexă și ar fi introdus o serie de provocări

legate de sincronizare și gestionarea timpului. Reamintirile necesită monitorizarea și declanșarea la momente precise în timp, ceea ce ar fi complicat și ar fi crescut predispoziția la erori dacă ar fi fost integrate direct în nucleul Rasa.

Crearea unei intenții suplimentare, *reminder_notification*, în loc să trimitem direct mesajul de reamintire de la serverul de remindere către controlorul de mesaje, a fost aleasă pentru a valorifica funcționalitățile avansate ale nucleului Rasa, cu toate că introduce o oarecare complexitate inofensivă. Nucleul Rasa se ocupă de selectarea răspunsurilor predefinite și umanizarea (reformularea) acestora pentru a evita repetitivitatea și pentru a oferi o experiență de utilizare mai naturală. Trimiterea task-ului înapoi la Rasa pentru procesarea mesajului permite reformularea reamintirii, asigurându-ne că mesajul final este coerent și atractiv. Această abordare ne permite să evităm clonarea funcționalității de reformulare în alte locuri, păstrând și folosind centralizat această funcționalitate în nucleul asistentului personal, care este responsabil de trimiterea răspunsurilor la comenzi.

5.5. Componenta de Umanizare

O ultimă componentă crucială prezentată în arhitectura conceptuală este componenta de umanizare (Odin Humanizer), o componentă care dispune de trei servicii, fiecare având o sarcină necesară în aducerea la viață a asistentului personal. Aceste servicii rulează pe o mașină separată, dotată cu hardware necesar susținerii a două modele de limbaj mari (LLMs) și un model TTS complex. Funcțiile îndeplinite de aceste modele sunt:

1. Funcția de dialog și conversație cu utilizatorul: Acest serviciu folosește un model de limbaj mare pentru a facilita o conversație naturală între utilizator și asistentul Odin.
2. Funcția de reformulare a răspunsurilor trimise de nucleul Rasa: Acest serviciu utilizează un alt model de limbaj mare pentru a reformula răspunsurile generate de nucleul Rasa, pe baza stării de spirit a bot-ului, asigurându-ne că eliminăm natura repetitivă a răspunsurilor predefinite.
3. Funcția de conversie a răspunsului sub formă de text în voce umană: Acest serviciu folosește un model TTS complex, care poate clona orice voce, pentru a converti răspunsul din textul al nucleului, în voce umană.

5.5.1. Conversația Umană

Scopul modelului de conversație este de a facilita un dialog captivant și fără limite între utilizator și chatbot. Cu ajutorul acestei componente se vor acoperi două intenții cheie din nucleul Rasa, *conversation_starter* și *nlu_fallback*, ceea ce înseamnă că, în afara intențiilor desemnate pentru comenzi, Odin va conversa cu utilizatorul, eliminând varianta opririi interacțiunii pentru că bot-ul nu a înțeles exact intenția

utilizatorului. Odin este programat să apeleze serviciul de generare al dialogului atât în momentul în care este detectată intenția *conversation_starter*, care a fost desemnată să generalizeze frazele ce pot indica intenția unui dialog, cât și atunci când intrarea de la utilizator nu face niciun sens pentru nicio intenție, iar bot-ul este programat să prezică astfel *nlu_fallback*.

Alegerea unui model potrivit a fost destul de simplă, cu un scop clar în minte: maximul de parametri ce încap în 6GB VRAM. În contextul unui model de limbaj mare, parametrii sunt valorile numerice care definesc structura și funcționarea rețelelor neuronale, iar un număr cât mai mare de parametri este crucial pentru capacitatea modelului de a învăța din date și de a genera text precis și relevant, reflectând complexitatea și puterea modelului de a înțelege și procesa limbajul natural. În urma acestei analize și a experimentării cu multe modele, am ales GPT-Neo cu 1,8 miliarde de parametri, care se potrivește perfect pe o placă video cu 6GB VRAM și, de asemenea, permite o viteză ridicată la generarea de text. Acest model asigură că Odin poate oferi răspunsuri detaliate și naturale, menținând astfel o conversație fluentă și captivantă cu utilizatorul.

Procesul de reglare fină (fine-tuning) a modelului constă în găsirea unui set de date potrivit cazului nostru de utilizare, preprocesarea datelor și antrenarea modelului pe datele obținute. Dialogul este printre cele mai comune utilizări în domeniul LLMs, prin urmare găsirea unui set de date ce constă în conversații cotidiene dintre oameni nu a fost o sarcină grea.

Setul de date numit „*DailyDialog*” este o colecție de dialoguri în limba engleză de înaltă calitate, destinată dezvoltării și evaluării modelelor de înțelegere a limbajului natural și a sistemelor de dialog. Acesta conține 13.118 dialoguri, împărțite în trei subseturi: un set de antrenament cu 11.118 dialoguri și seturi de validare și testare, fiecare cu câte 1.000 de dialoguri. Fiecare dialog din setul *DailyDialog* are, în medie, aproximativ 8 schimburi de replici între vorbitori, fiecare replică conținând în jur de 15 tokeni.

Datele găsite au fost preprocesate într-un format adecvat pentru procesul de tokenizare, mai exact aranjarea lor într-un fișier CSV cu două coloane, în coloana *Input* sunt așezate rânduri de dialog dintre două persoane separate prin ‘\n’, iar în coloana *Output* se află rândul de dialog ce trebuie prezis de către model în urma conversației precedente din coloana *Input*. Pentru procesul de tokenizare, discutat în secțiunea 5.3.1, datele de pe cele două coloane sunt agregate într-un singur șir, separate printr-un token special, în cazul nostru `<|endoftext|>`, care are rolul de a semnala modelului, în procesul de antrenare, momentul în care trebuie să înceapă generarea de text. Fine-tuning-ul în sine va avea loc pe platforma DGX a facultății, care dispune de hardware necesar susținerii unui asemenea proces, deoarece reglarea fină a unui model necesită mult mai mult VRAM decât inferența cu acesta (generarea de text), dar mai multe detalii de implementare a procesului vor fi prezentate în capitolul următor.

Serviciul de conversație constă în crearea unui nou endpoint POST cu ajutorul microframework-ului Flask, la destinația */dialogue*, care are rolul de a genera dialog pe baza parametrului *input_text* trimis din nucleul Rasa de către acțiunea *action_generate_dialogue*, ce simbolizează intrarea de la utilizator. Modul de încărcare și generare a modelului, din păcate, este „*stateless*”, adică nu ține cont de ce s-a generat anterior, fapt ce nu ne avantajează în încercarea de a construi un bot cu capabilități de conversație umană, conștientă de context. Pentru a imita un model „*stateful*”, contextul este construit în nucleu sub forma unui șir de caractere, structurat similar cu datele de antrenare a modelului, pentru ușurința recunoașterii rândurilor de dialog dintre utilizator și bot.

5.5.2. Reformularea Bazata pe Emoții

Reformularea răspunsurilor predefinite din Rasa bazate pe starea de spirit a botului este una din cele mai importante, dacă nu cea mai importantă parte în procesul de umanizare a asistentului Odin, deoarece elimină natura repetitivă a răspunsurilor predefinite ale unui chatbot construit în Rasa și dă o notă de naturalețe și variabilitate a conversațiilor. Acest proces permite asistentului să răspundă în moduri diferite la aceeași întrebare, în funcție de contextul conversației și de starea de spirit atribuită botului, cum ar fi bucurios, neutru sau supărat.

Noile intenții din Rasa asociate stării de spirit sunt *good_intent*, *bad_intent* și *apology* și se referă la emoțiile transmise de utilizator în mesajul său, precum „*I love you, Odin.*”, „*Odin, you are useless.*” sau „*I’m sorry for what I have said before.*”. Asociate acestor intenții avem slot-ul *mood* ce poate lua valori din setul $(-1, 0, 1)$, reprezentând stările de spirit *angry*, *neutral* și respectiv *happy*. La detectarea intențiilor *good_intent* sau *bad_intent* se declanșează acțiuni care incrementează, respectiv decrementează slot-ul de *mood* cu 1, iar la intenția *apology*, asistentul acceptă scuzele și setează *mood*-ul la 0 în cazul în care era setat la -1 înainte.

Modelul ales pentru acest task este din nou GPT-Neo cu 1.8 miliarde de parametri, însă procesul de procurare a datelor a reprezentat o provocare majoră. Dat fiind un caz de utilizare unic, sau cel puțin nu există documentări asociate lui pe internet, am optat să generez setul de date singur, cu ajutorul unui LLM suficient de performant încât să înțeleagă posibilele tonalități subtile care intra în procesul de reformulare pe baza emoțiilor, cum ar fi GPT-4o.

Construirea unui set de date de acest gen constă în generarea unei liste de fraze care să reprezinte răspunsurile posibile ale unui asistent personal la diverse comenzi, apoi reformularea acestor răspunsuri pe un ton fericit, neutru sau nervos, cu ajutorul lui GPT-4o. În acest sens, fiind nevoie de un set de date mic în comparație cu majoritatea seturilor de date folosite pentru antrenare sau reglare fină, dar totuși mare pentru generarea sa ineficientă cu „copy-paste” din browser într-un fișier .csv, am optat pentru generarea sa cu ajutorul unui script în Python care integrează API-ul GPT-4o, cel mai performant și optim în raportul preț/calitate dintre modelele puse la dispoziție de

OpenAI. Astfel am creat o serie de prompturi de generare și reformulare care au fost folosite în combinație pentru realizarea setului de date.

În primul rând, în prompturile de generare este nevoie de un prompt global care să ofere contextul corespunzător modelului generator, în sensul în care propozițiile generate să fie structurate ca un răspuns dat de un asistent personal. Promptului global îi se adaugă și un prompt de generare, care face parte dintr-o familie de prompturi orientate spre generarea răspunsurilor asistentului la intențiile definite deja în Rasa.

Un al doilea pas, după generarea răspunsurilor, este reformularea fiecăruia în parte, unde din nou intervine o serie de trei prompturi pentru fiecare mood posibil (*happy*, *neutral*, *angry*), detaliate în așa fel încât să capteze particularitățile semantice dorite în reformulări. În acest sens promptul de reformulare pe baza mood-ului ‘happy’ specifică inserarea pozitivității și bucuriei în răspunsul reformulat, iar cel aferent mood-ului *angry* încurajează reformularea cu un ton sarcastic, sictirit și neprietenos. Aceste trei instrucțiuni de reformulare sunt din nou însoțite de un prompt global, care se asigură că modelul generator înțelege contextul reformulării, asigurând astfel un set de date de calitate.

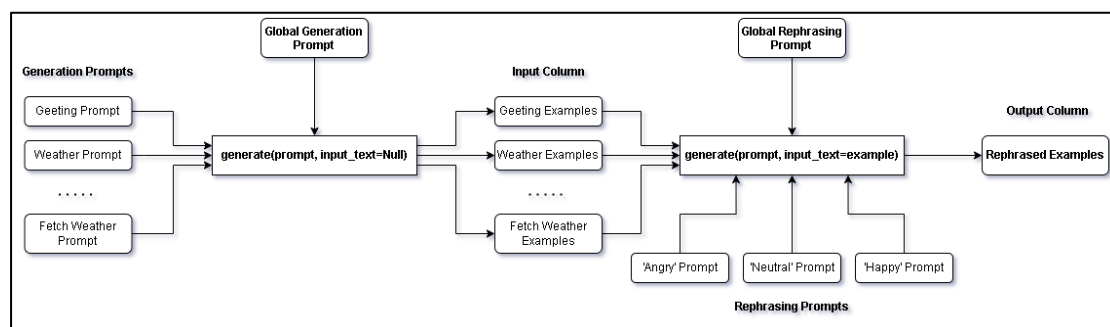


Figura 9: Schema Generării Setului de Date

Un exemplu bun în acest caz ar fi răspunsul predefinit de la acțiunea discutată anterior *action_fetch_weather*, anume „Cluj – 25th of July 2024, 5:00 PM: 31°C, sunny, 5 km/h wind” ar putea fi reformulat într-o manieră fericită: „Enjoy this sunny afternoon in Cluj, with a temperature of 31°C and a gentle breeze at 5 km/h”; sau într-o manieră nervoasă și sarcastică: „Enjoy the scorching heat in Cluj on the afternoon of 25th of July, make sure not to wear sun cream.”. Având această abordare, modelul ar trebui să învețe pe lângă particularitățile fiecărei stări de spirit, precum notele de bucurie și voie bună ale mood-ului *happy*, și tentele de sarcasm introduse de mood-ul *angry*. Modelul de asemenea ar trebui să învețe să păstreze informațiile relevante din context între textul primit și textul reformulat. Procesul de fine-tuning este similar cu cel al modelului anterior, agregarea tuturor datelor relevante și separarea acestora prin tokeni distincți într-un singur șir de caractere, sub forma, urmată de tokenizarea acestora. Similar, am folosit platforma DGX, dar cu mici modificări în script la parametrii de antrenare și cei de generare.

Serviciul de reformulare, asemănător cu serviciul de conversație, constă în crearea unui endpoint cu Flask la adresa */rephrase*, care primește ca parametri textul de reformulat, intenția detectată în comanda utilizatorului și starea de spirit a bot-ului, și reformulează pe baza acestora răspunsul standard al bot-ului. Înainte ca informațiile să fie tokenizate și trimise în model pentru generare, acestea sunt agregate într-un format identic cu cel folosit în datele de antrenament, pentru performanțe cât mai ridicate. Odată primit textul, modelul GPT-Neo procesează intrarea și generează o reformulare care reflectă starea de spirit specificată.

5.5.3. Conversia Text-Vorbire (Text-to-Speech)

O ultimă componentă a bot-ului, care rulează pe hardware-ul dedicat și necesită o cantitate mare de VRAM, este modelul XTTS v2, folosit pentru a converti răspunsul lui Odin din text în voce umană. Modelul este extrem de performant și capabil, având posibilitatea de a clona voci într-un mod „0-shot”, adică modelul nu este antrenat și nu a mai auzit vocea respectivă, și o primește odată cu textul de convertit.

Modelul XTTS necesită mai puțin RAM decât restul modelelor și este încărcat în memorie și folosit „out of the box”, fără alte modificări ulterioare. Totuși, consumul său de resurse este încă prea mare pentru a fi rulat local, astfel că, odată cu introducerea acestuia în pipeline, resursele hardware de pe serverul dedicat componentei de umanizare sunt utilizate la capacitate maximă. De asemenea, se păstrează și o clonă a vocii mele, o înregistrare de 6 secunde, pentru a-i oferi un glas provizoriu asistentului personal.

Implementarea serviciului, similar cu cele de până acum, implică un endpoint POST în Flask la adresa */synthesize*, care primește ca parametru textul ce urmează a fi convertit și este invocat de această dată din controlorul de mesaje, după tot pipeline-ul de procesare și reformulare a comenzii. Conversia TTS este ultima componentă activă din tot procesul distribuit de procesare a cererilor utilizatorului.

Această componentă introduce o latență semnificativă în dialog. Chiar dacă conversia TTS este relativ rapidă, timpul necesar pentru a procesa textul și a genera vocea, combinat cu latența introdusă de celelalte componente ale sistemului, se adună într-un timp relativ mare, de câteva secunde, pentru a oferi un răspuns complet. Acest aspect poate afecta fluiditatea conversației în timp real, însă este un compromis necesar pentru a asigura o calitate superioară a interacțiunii vocale.

5.6. Comunicarea între Componente

În arhitectura sistemului, toate componentele comunică între ele prin metoda POST HTTP. Aceasta metodă a fost aleasă pentru a facilita trimiterea și primirea mesajelor aflate într-un anumit stadiu de procesare și pentru a aștepta răspunsul în urma

procesării mesajului. Prin utilizarea cererilor POST secvențiale de la o componentă la alta, se asigură un flux ordonat și clar al datelor.

5.6.1. Fluxul de comunicare între componente

1. Chat Controller la nucleul asistentului (Rasa): Procesul începe cu Chat Controller, care primește comanda text introdusă de utilizator și trimite un POST request cu aceasta către nucleul Rasa la adresa *localhost:5005/webhooks/rest/webhook*. Aici, comanda este procesată pentru a determina intenția utilizatorului și a extrage eventualele entități.
2. Nucleul Rasa la serverul de acțiuni: După procesarea comenzii, nucleul Rasa decide ce acțiune trebuie executată. Pentru acțiunile personalizate, nucleul trimite un POST request către serverul de acțiuni, care rulează separat la adresa *localhost:5055*. Acesta se ocupă de executarea logicii specifice acțiunii cerute.
3. Serverul de acțiuni la API-uri externe: Serverul de acțiuni poate trimite cereri către API-uri externe pentru a obține date necesare. De exemplu, pentru o acțiune de verificare a vremii, sau programarea unui reminder.
4. Serverul de acțiuni la componenta de umanizare: După obținerea datelor externe, serverul de acțiuni trimite un POST request către componenta de umanizare, expusă la adresa *192.168.0.107:5002/rephrase* sau *192.168.0.107:5001/dialogue*, în funcție de necesitatea reformulării sau începerea unei conversații.
5. Componenta de umanizare răspunde înapoi la serverul de acțiuni: Componenta de umanizare procesează cererea și trimite răspunsul înapoi la serverul de acțiuni, care este integrat și strâns legat cu nucleul Rasa și redirecționează apoi răspunsul către acesta.
6. Nucleul Rasa răspunde înapoi la Chat Controller: Nucleul Rasa primește răspunsul de la serverul de acțiuni, finalizează procesarea și trimite răspunsul final înapoi la Chat Controller.
7. Chat Controller apelează componenta de umanizare pentru conversia TTS: În cazul în care răspunsul trebuie convertit în voce, Chat Controller trimite încă o dată un POST request componentei de umanizare, la *192.168.0.107:5003/synthesize* pentru a realiza conversia text-vorbire. Răspunsul audio este apoi redat utilizatorului.

Este important de observat că tot fluxul de procesare și răspuns al unei comenzi intră și iese din nucleul Rasa, după cum se observa din schema generală a sistemului în **Figura 5**. Acest lucru se aplică chiar și mesajelor de reamintire. Această abordare evită legăturile ciclice între componente și asigură că toate mesajele intră și ies din miezul

asistentului. Acest design centralizat simplifică gestionarea fluxului de date și permite o mai bună monitorizare și control al întregului sistem.

5.6.2. VPN-ul

Asistentul personal pe care îl dezvoltăm are componente care rulează local, pe laptop și componente care rulează pe un hardware dedicat acasă, denumite în proiect componentele de umanizare. Este de notat faptul că de-a lungul lucrării, am menționat în multe locuri apeluri către componentele de umanizare la IP-ul *192.168.0.107*, care este un IP local, dar totuși aceste apeluri sunt permise și primesc răspuns indiferent de rețeaua în care se afla laptop-ul, atâta timp cât are acces la internet. Acest fapt este posibil deoarece comunicarea dintre aceste componente se face printr-o rețea privată virtuală (VPN), folosind *OpenVPN*. Acest setup permite accesul securizat la endpoint-urile de pe hardware-ul dedicat (*192.168.0.107*) chiar și atunci când nu suntem în rețeaua locală, de acasă.

Serverul OpenVPN este instalat și configurat pe routerul de acasă, acesta creează și gestionează conexiunile VPN. Configurarea serverului implică generarea unor chei de criptare și a unui fișier de configurare care conține parametrii necesari pentru stabilirea conexiunii.

Clientul OpenVPN este instalat pe laptopul local. Clientul utilizează un fișier de configurare (furnizat de serverul OpenVPN) care conține cheile de criptare și alte setări necesare pentru conectarea la server.

Pentru inițierea unei conexiuni către serverul OpenVPN, clientul OpenVPN utilizează fișierul de configurare primit de la server. Procesul de autentificare utilizează cheile de criptare pentru a asigura că doar dispozitivele autorizate pot accesa rețeaua. Odată ce conexiunea este stabilită, tot traficul de date dintre laptopul local și hardware-ul dedicat este tunelat prin VPN. Aceasta înseamnă că datele sunt criptate pe tot parcursul drumului, asigurându-se astfel securitatea și integritatea informațiilor transmise. Din perspectiva laptopului local, hardware-ul dedicat apare ca și cum ar fi în aceeași rețea locală. De aceea, putem accesa endpoint-ul *192.168.0.107* chiar și de la distanță, fără a fi nevoie să deschidem un port în zona demilitarizată (eg. Demilitarized Zone, DMZ) a routerului, prin care să fie accesată componenta de umanizare. Un aspect interesant al utilizării OpenVPN este faptul că, odată configurat, pe laptopul local apare o nouă placă de rețea virtuală. Aceasta este instanțiată de OpenVPN și este utilizată pentru a imita o conexiune la nivel de rețea locală cu routerul. Placa de rețea virtuală permite dispozitivului local să comunice direct cu rețeaua de acasă ca și cum ar fi fizic prezent în acea rețea.

Această abordare este mult mai sigură decât folosirea DMZ datorită posibilității criptării traficului, făcându-l mult mai dificil de interceptat sau atacat de către terți. Pe lângă acest aspect, deschiderea unui port în DMZ ar expune direct serverul la internet, crescând riscul de atacuri cibernetice.

Capitolul 6. Testare și Validare

În acest capitol se va facilita atât testarea separată a componentelor bot-ului și a algoritmilor propuși, cât și testarea integrării acestora, evaluând astfel capabilitățile asistentului Odin. Dat fiind un astfel de sistem, este dificil de evaluat adevărata performanță, deoarece nu există date de test sau metrice disponibile pe internet, iar acestea trebuie să fie create manual. Prin urmare, compararea soluției curente cu alte soluții existente devine complicată, cea mai relevantă metrică în acest caz fiind experiența utilizatorului.

Date fiind aceste circumstanțe, generarea testelor de către dezvoltatorul aplicației poate implica subiectivitate în sensul că testele sunt meticulos construite sau soluția este adaptată în funcție de teste, în așa fel încât scorul să fie cât mai mare. Astfel, capitolul va fi alcătuit din trei mari obiective în care toate testele propuse vor fi lipsite de subiectivitatea dezvoltatorului pe cât de mult posibil: testarea unitară, evaluarea datelor de antrenare a modelelor de limbaj folosite în componentele de umanizare și testarea experienței utilizatorilor cu întregul sistem.

6.1. Testarea Nucleului

Nucleul asistentului personal este componenta centrală responsabilă pentru înțelegerea limbajului natural, selectarea răspunsurilor, executarea acțiunilor și facilitarea comunicării între celelalte componente ale asistentului. Evaluarea acestei componente este esențială pentru a asigura performanța și fiabilitatea asistentului Odin. În continuare, vom evalua componentele și algoritmii pentru care este posibilă și are sens crearea manuală a unor teste, în afara testării bazate pe experiența utilizatorului.

6.1.1. Clasificarea Intențiilor

Înțelegerea limbajului natural de către nucleul asistentului implică două sarcini cruciale: clasificarea corectă a intențiilor și extragerea entităților potrivite asociate intențiilor. Testarea acestor sarcini este esențială pentru a asigura funcționarea corectă și eficientă a asistentului personal Odin.

Primul caz de test realizat a fost cel al clasificării intențiilor. Pentru a păstra obiectivitatea datelor de test, s-a folosit ChatGPT pentru generarea a câte 10 exemple pentru fiecare din cele 16 tipuri de intenții existente pe care a fost antrenat nucleul. Toate exemplele generate au fost noi și diferite față de cele existente în setul de date de antrenament, asigurând astfel o testare corectă și relevantă. Framework-ul Rasa, fiind unul complet, simplifică testarea NLU a nucleului printr-o simplă comandă în terminal: „`rasa test nlu --nlu {path_to_file}`”. Rezultatele testelor includ metrice precum

acuratețea, precizia, recall-ul și F1-score-ul pentru fiecare intenție, oferind o imagine clară asupra performanței clasificatorului. Metricile discutate în continuare au fost definite și teoretizate anterior, în capitolul 4.4. În urma testării NLU a nucleului Rasa, raportul generat prezintă următoarele rezultate: acuratețea de 93.1%, precizia de 94.9%, recall-ul de 93.1% și scorul F1 de 93.1%, reflectând un echilibru optim între precizie și recall.

Aceste rezultate au fost obținute pe un set de 160 de exemple generate obiectiv de ChatGPT, diferite de datele utilizate pentru antrenament. Acuratețea de 93% indică faptul că modelul de clasificare a intențiilor funcționează foarte bine, depășind standardul acceptabil în industrie de 70%-90%. Precizia, recall-ul și F1-score-ul sunt toate la niveluri foarte ridicate, sugerând că modelul nu doar clasifică corect intențiile, dar și că identifică majoritatea cazurilor relevante (recall) și minimizează predicțiile false pozitive (precizie).

Cu o acuratețe de 93.1%, modelul se situează peste standardul 70%-90%, demonstrând astfel că nucleul Rasa este bine antrenat și capabil să înțeleagă corect intențiile utilizatorilor. Totuși, este important de reținut că acuratețea în domeniul învățării automate este subiectivă și depinde de multe variabile, inclusiv de complexitatea datelor de intrare și de variabilitatea acestora. Deși metricile de performanță ale modelului sunt impresionante, cea mai relevantă metodă de evaluare rămâne testarea experienței utilizatorului. Aceasta implică evaluarea integrată a tuturor componentelor sistemului în cadrul asistentului Odin și observarea modului în care acestea funcționează împreună pentru a oferi o experiență fluentă și eficientă.

Matricea de confuzie prezentată în Figura **Eroare! Fără sursă de referință.** include următoarele elemente: Axe X reprezintă etichetele prezise, în timp ce axe Y reprezintă etichetele reale. Fiecare celulă conține un număr care indică frecvența unei perechi de etichete reale și prezise. De exemplu, celula (1, 1) arată de câte ori intenția "apology" a fost corect clasificată ca "apology". Numerele de pe diagonala principală indică numărul de predicții corecte pentru fiecare intenție. De exemplu, numărul "10" pe celula (1, 1) arată că toate cele 10 exemple pentru intenția "apology" au fost corect clasificate. Numerele din celulele non-diagonale indică numărul de exemple care au fost clasificate incorect. De exemplu, celula (3, 1) indică faptul că 4 exemple pentru intenția "bad_intent" au fost incorect clasificate ca "apology".

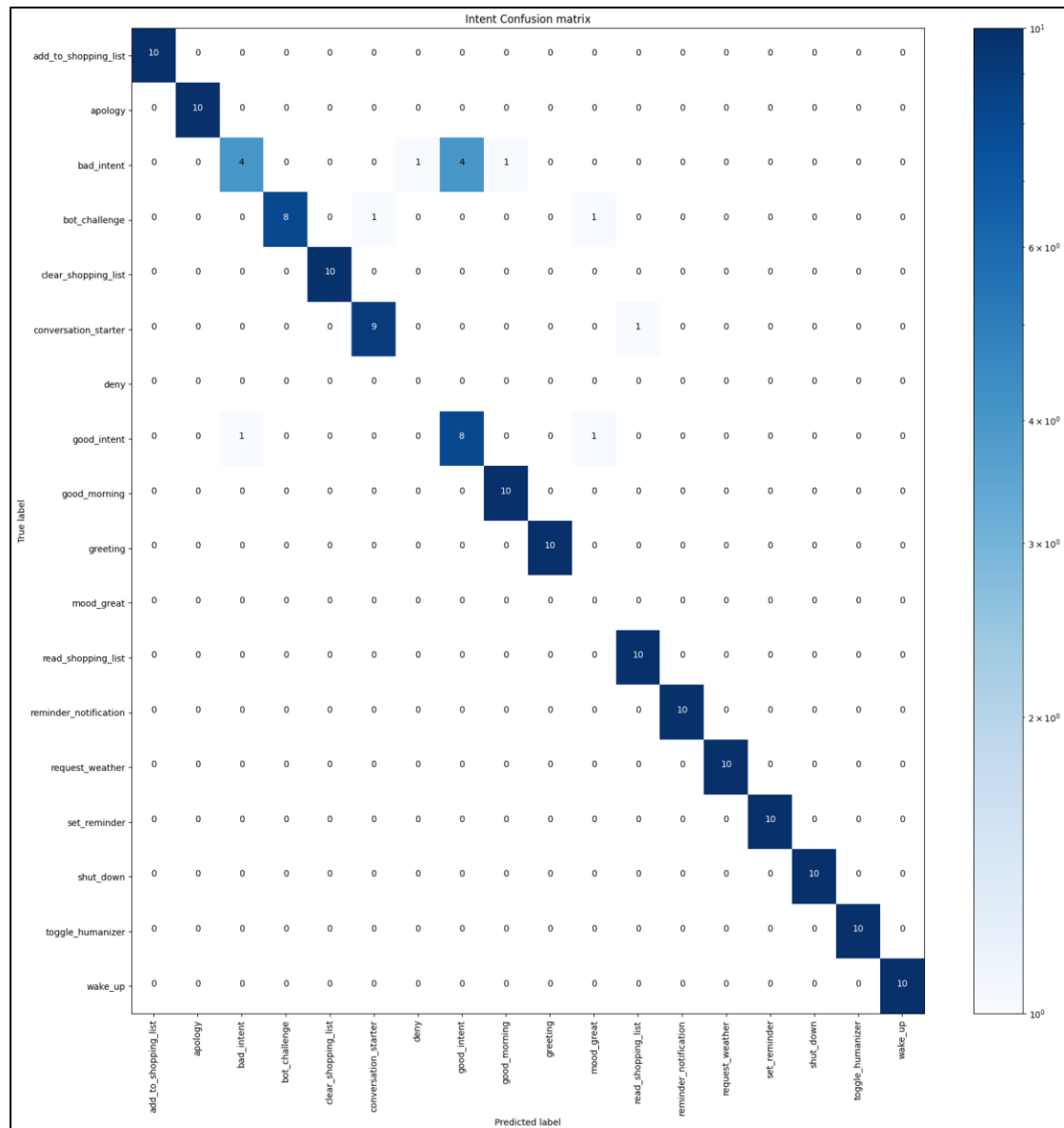


Figura 10: Matricea de Confuzie a Intențiilor

Precizia mare pentru majoritatea intențiilor este evidentă, diagonală principală a matricei arătând un număr mare de predicții corecte pentru majoritatea intențiilor, indicând o performanță bună a modelului. De exemplu, intențiile *add_to_shopping_list*, *apology*, *good_morning* și *wake_up* au toate 10 predicții corecte. Totuși, observăm câteva celule non-diagonale care indică erori de clasificare. De exemplu, intenția *bad_intent* a fost clasificată greșit de 4 ori ca *apology* și de 4 ori ca *good_intent*. Acest lucru sugerează că modelul nostru are dificultăți în a distinge între aceste intenții specifice, posibil datorită similarității în datele de intrare. Zonele cu erori frecvente, cum ar fi confuzia dintre *bad_intent* și *good_intent*, sugerează că setul de date de antrenament ar putea fi îmbunătățit prin adăugarea mai multor exemple distinctive. Matricea de confuzie oferă o validare detaliată a modelului, arătând nu doar acuratețea generală, ci și specificul punctelor tari și slabe ale clasificatorului.

Histograma prezentată în **Figura 10** vizualizează nivelul de încredere al predicțiilor făcute de modelul de clasificare a intențiilor. Bara din stânga reprezintă predicțiile corecte, afișate în albastru, în timp ce bara din dreapta reprezintă predicțiile incorecte, afișate în roșu. Pe axa verticală se găsesc valorile de „confidență” sau încredere, care variază de la 0 la 1, iar pe axa orizontală se află numărul de exemple care au fost prezise corect sau incorect la diferite nivele de încredere.

Histograma arată că majoritatea predicțiilor corecte ale modelului au niveluri de confidență foarte ridicate, cele mai multe fiind concentrate în intervalul 0.97-1.00. Acest lucru sugerează că atunci când modelul este sigur de predicția sa, este, în general, corect. În schimb, predicțiile incorecte tind să aibă niveluri de confidență mai variate. Acest lucru indică faptul că modelul face greșeli atunci când este mai puțin sigur de predicția sa.

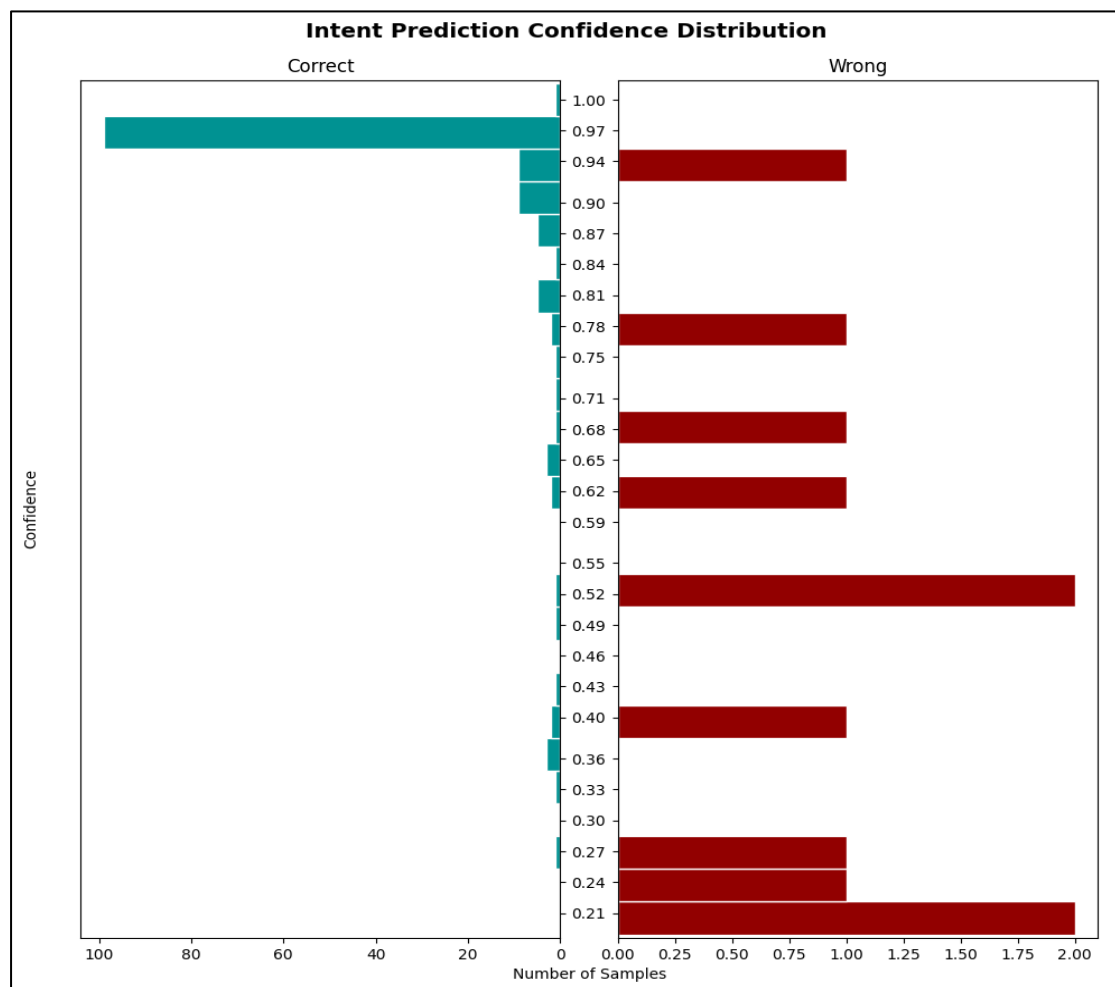


Figura 11: Histograma de Predicție a Intențiilor

Îmbunătățirea performanței modelului, ar însemna ridicarea barelor albastre și pe reducerea barelor roșii. Acest lucru poate fi realizat prin îmbunătățirea calității datelor de antrenament, adăugând mai multe exemple relevante și diverse pentru fiecare intenție, și ajustând hiperparametrii modelului. Reducerea numărului de bare roșii indică un model mai robust și mai sigur pe predicțiile sale, crescând astfel acuratețea generală și încrederea utilizatorilor în asistentul personal Odin.

6.1.2. Extragerea Entităților

Următoarea componentă crucială în înțelegerea limbajului natural este cea de extragerea a entităților. Aceasta implică identificarea și izolarea unor elemente specifice din text care au o semnificație particulară, cum ar fi nume, date, locații sau alte tipuri de informații relevante. În contextul asistentului personal Odin, vom supune testelor doar intențiile care au entități asociate, cum ar fi *request_weather*, *set_reminder* și *add_shopping_list*.

Pentru a asigura consistența și obiectivitatea testelor, datele de testare pentru extragerea entităților vor fi aceleași ca și la pasul de clasificare a intențiilor. Singurele exemple care vor fi luate în considerare vor fi acelea care dispun de entități. În acest sens, rulând comanda „**rasa test nlu --nlu {path_to_nlu}**”, framework-ul Rasa va genera toate informațiile relevante necesare pentru evaluarea comportamentului modelului de extragere a entităților, DIETClassifier.

Raportul generat arată performanța foarte bună a clasificatorului DIET în extragerea entităților. Precizia de 98.54% indică faptul că majoritatea entităților extrase sunt corecte. Recall-ul de 91.24% arată că modelul identifică o proporție mare din toate entitățile prezente în setul de date, iar scorul F1 de 93.83% sugerează un echilibru bun între precizie și recall. Acuratețea de 98.36% reflectă că majoritatea entităților din întregul set de date au fost identificate corect.

Matricea de confuzie prezentată pentru extragerea entităților oferă o imagine clară asupra performanței modelului DIETClassifier în această sarcină. Interpretarea unei astfel de matrici a fost descrisă conceptual la capitolul 4.4, și exemplificată anterior la sub-sectiunea 6.1.1 pentru clasificarea intențiilor.

Un aspect important de menționat este prezența entității *no_entity*. Aceasta nu este propriu-zis o entitate, ci reprezintă cazurile în care cuvintele din exemplele setului de date de testare nu sunt considerate entități. Acest lucru explică numărul ridicat de instanțe (717) din coloana și rândul asociat *no_entity*.

Prezența altor numere pe coloana *no_entity* semnifică faptul că unele entități existente nu au fost identificate și au fost clasificate ca cuvinte simple sub denumirea de *no_entity*. De exemplu, *city* a fost uneori clasificat greșit ca *no_entity*, ceea ce înseamnă că entitatea *city* trebuia identificată, dar modelul a determinat că nu există nicio entitate în text. Similar, entitatea *offset_time* a fost uneori confundată cu *no_entity*, indicând că modelul ar putea avea dificultăți în identificarea precisă a valorilor de timp într-un context specific.

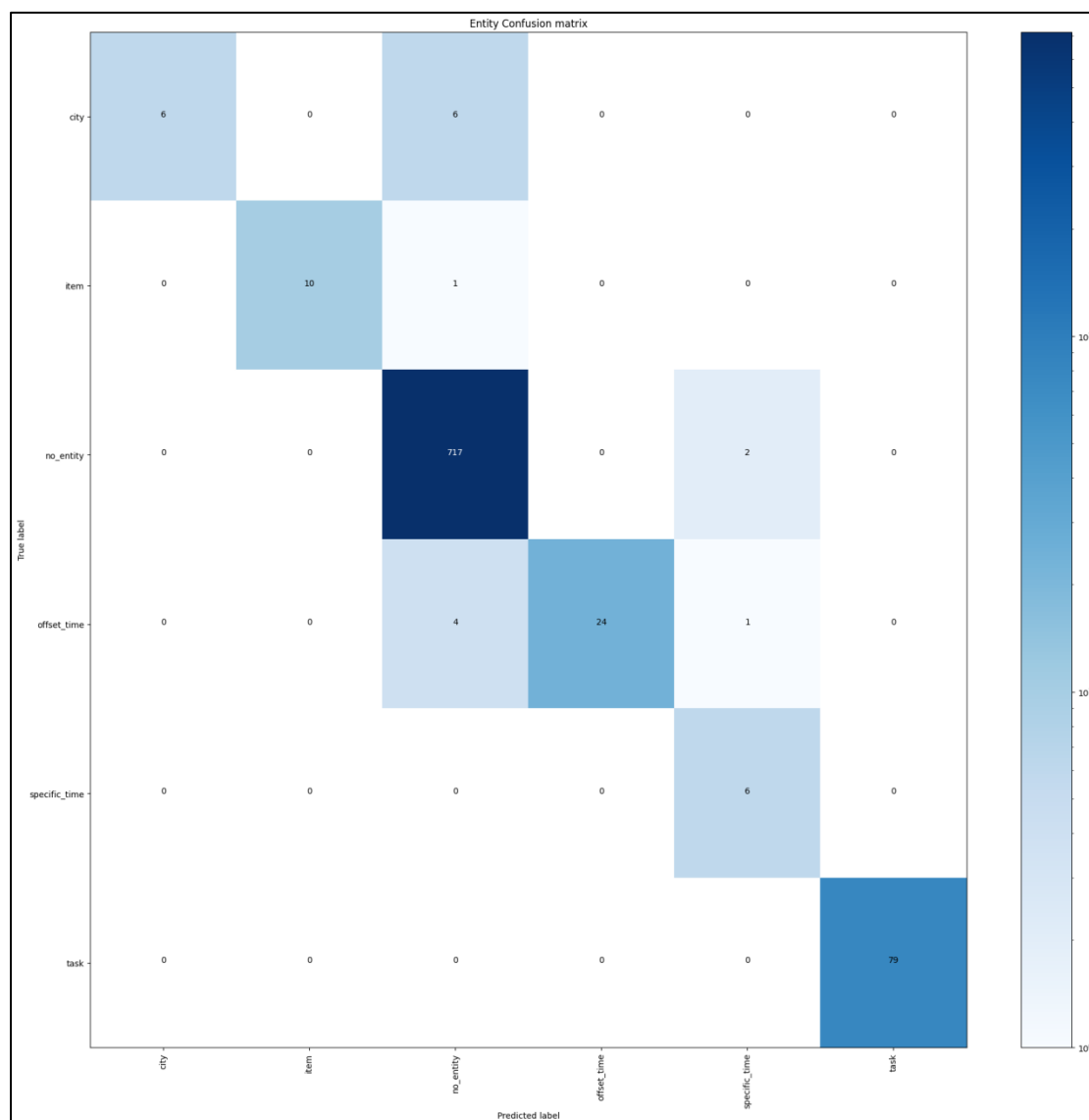


Figura 12: Matricea de Confuzie a Entităților I

În **Figura 12**, Observăm că numărul considerabil mai mare de apariții ale entității *no_entity* față de celelalte entități relevante influențează puternic acuratețea modelului, rezultând într-un scor de 98%. Această influență semnificativă face ca acuratețea reală a extragerii entităților să fie umbrită. Prin urmare, este esențial să separăm intențiile ce conțin entități într-un set de date distinct și să testăm din nou modelul. Acest lucru ne va permite să evaluăm mai precis performanța modelului în ceea ce privește identificarea corectă a entităților relevante și să ajustăm setul de date sau parametrii modelului pentru a îmbunătăți acuratețea generală în aceste cazuri.

În urma separării datelor relevante care conțin entități și reiterării testului, acuratețea a scăzut la doar 95%. În noua matrice de confuzie, observata în **Figura Eroare! Fără sursă de referință.**, numărul cuvintelor clasificate ca *no_entity* a scăzut la 150. Observăm că instanțele de predicție greșită ale modelului sunt în cea mai mare parte ratări complete ale entităților, nu confuzii între ele. Entitatea *city* este identificată

corect cu un scor f1 de 0.66, ceea ce este semnificativ mai mic decât acuratețea generală, dar rămâne totuși acceptabil.

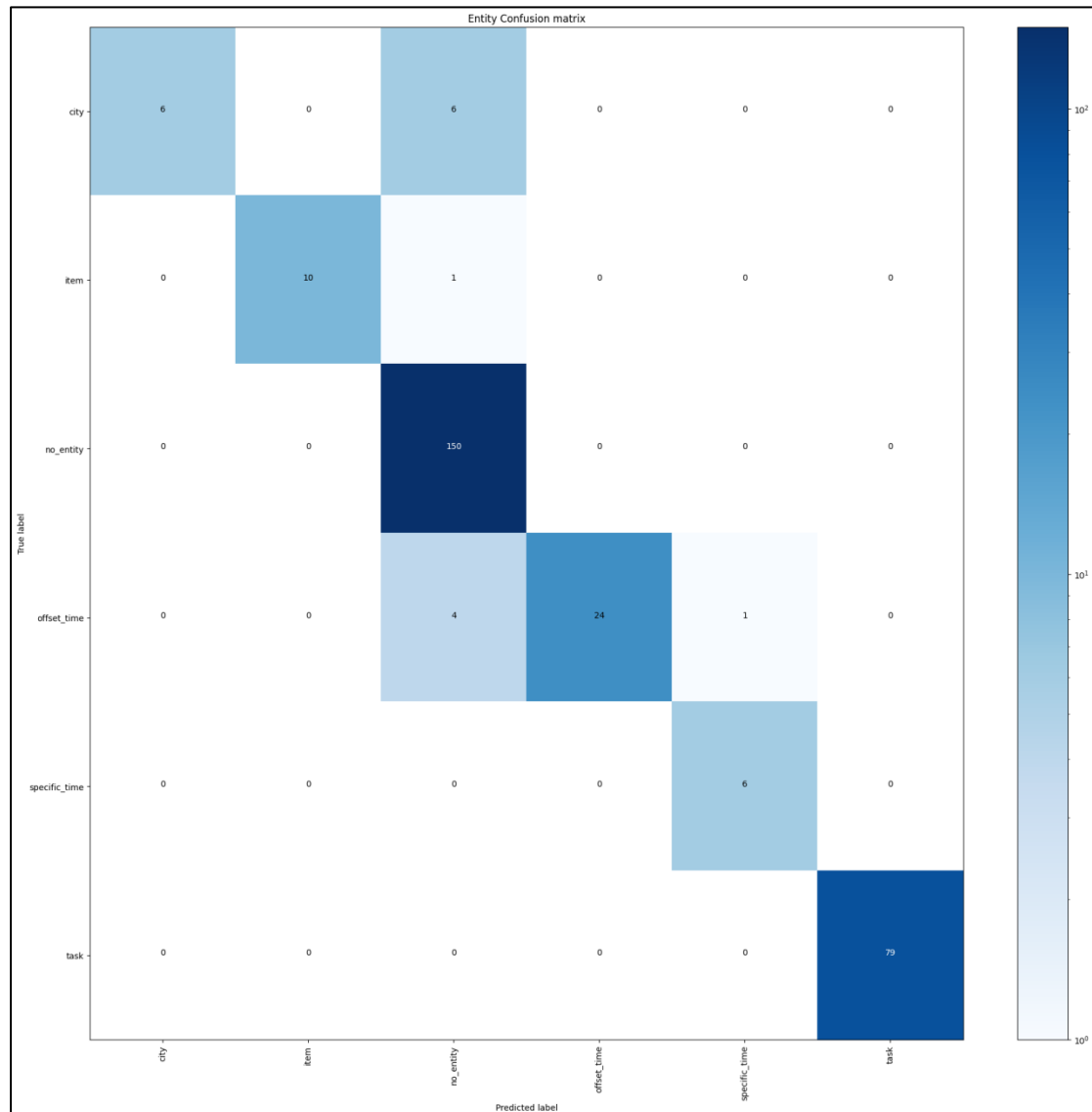


Figura 13: Matricea de Confuzie a Entităților II

6.2. Testarea Algoritmilor Ajutători

Există anumite acțiuni care folosesc tehnici suplimentare de procesare a entităților și apeluri către API-uri externe, precum *action_fetch_weather* sau *action_set_reminder*. Ar avea sens testarea logicii ajutoare apelate în realizarea acestora, pentru a ne asigura că răspunsurile finale construite în acțiuni sunt de bună calitate. În acest caz cea mai potrivita metoda de testare ar fi testarea unitara (unit testing), care evaluează funcționarea individuala și izolată a componentelor într-un mediu de test predefinit.

6.2.1. Convertorul de Timp

De exemplu, acțiunea *action_fetch_weather* implică mai multe etape de prelucrare a datelor. Mai întâi, entitățile extrase din comanda utilizatorului, cum ar fi orașul și timpul, sunt procesate pentru a construi o interogare către API-ul *OpenWeatherMap*. Similar cu programarea unui reminder, algoritmul trebuie să transforme entitățile de timp din limbaj natural într-un format specific *datetime* acceptat de API.

Structura generală a unui test pentru validarea funcționalității de procesare a specificărilor de timp în limbaj natural implică mai multe etape. În primul rând, este generată o specificare a timpului în limbaj natural, cum ar fi „*in 10 minutes*” sau „*next week*”. Apoi, se creează un obiect numit *expected*, care se incrementează corespunzător și reprezintă timpul absolut care ar trebui extras din limbajul natural. Este important să setăm componentele de timp irelevante din obiectul *datetime* la zero pentru a asigura trecerea corectă a testelor.

De exemplu, în cazul expresiei „*in 10 minutes*”, valoarea *expected* va avea minutele ajustate conform specificării, dar secunde și milisecunde vor fi setate la zero. În mod similar, pentru expresia „*next week*”, ora, minutul, secunda și milisecunda vor fi setate la zero, deoarece nu sunt relevante pentru intervalul de timp specificat. Datele de test au fost generate cu ajutorul ChatGPT, având grijă să fie acoperite toate expresiile regulate după care caută algoritmul în expresie. Astfel, au fost create exemple care includ expresii regulate combinate pentru a crește dificultatea testelor.

Au fost generate 13 cazuri de test cu dificultăți variabile în așa fel încât să se verifice robustețea algoritmului, dintre care au trecut 12. Singurul test picat a fost la exemplul „*two days from now*”, deoarece algoritmul așteaptă șablonul „*in ... from now*” pentru a putea identifica expresia regulată corect, dar într-adevăr testul eșuat este valid, și a semnalat un caz care a fost omis în procesul de implementare.

6.2.2. OpenWeatherMap API

Testarea corectitudinii și fiabilității rezultatelor obținute de la API-uri externe este esențială pentru a asigura calitatea răspunsurilor generate de asistentul personal. Dat fiind faptul că acțiunea *action_fetch_weather*, se bazează pe date obținute din exterior pentru a formula răspunsuri corecte, este important să ne asigurăm că aceste date sunt gestionate corect. În acest sens, folosim *unittest.mock* pentru a crea teste simulate care verifică funcționarea corectă a apelurilor API.

Mock-ul ne permite să simulăm comportamentul cererilor HTTP fără a face efectiv cereri reale. Acest lucru economisește timp și resurse și asigură că testele noastre sunt determinate și reproductibile. Prin configurarea mock-ului, putem defini ce răspunsuri ar trebui să returneze *requests.get*, permițându-ne să testăm cum gestionează funcțiile noastre aceste răspunsuri.

Am creat un set de 4 teste relevante, iar fiecare metodă din această clasă reprezintă un caz specific de testare. Pentru a simula răspunsurile API-ului, utilizăm `unittest.mock.patch`, care înlocuiește temporar funcțiile specifice, în acest caz `requests.get`, cu obiecte mock. Aceste obiecte mock sunt configurate pentru a returna răspunsuri prestabilite sau pentru a ridica excepții, în funcție de scenariul de testare.

Pentru a acoperi toate cazurile, testele verifica atât cazurile de succes în care parametrul `city` reprezintă un oraș valid, cât și cazurile în care acest parametru este invalid, precum „*InvalidCity*”, iar cererea HTTP ar trebui să eșueze. De asemenea se testează returnarea corectă a datelor precum și validitatea și sensul acestora. Toate 4 cazurile de test au trecut, indicând că răspunsurile de la API-ul meteo sunt consistente și corecte.

6.3. Evaluarea Modelelor de Limbaj

În acest subcapitol vom revizui și interpreta rezultatele proceselor de fine-tuning la care au fost supuse modelele de limbaj mari (LLM-uri) utilizate în componenta de umanizare a asistentului personal Odin. În cazul nostru, modelele de limbaj folosite în componenta de umanizare a asistentului Odin au fost supuse unui proces de fine-tuning pentru a le face capabile să reformuleze răspunsurile în funcție de starea de spirit a asistentului, precum și pentru a facilita conversații naturale și coerente cu utilizatorul, prin urmare s-au folosit seturi de date foarte diferite structural pentru două cazuri de utilizare la fel de diferite, acestea fiind descrise în detaliu în capitolele 5.5.1 și 5.5.2. Mai departe, se vor prezenta datele extrase în urma proceselor de fine-tuning sub forma unor grafice, care pe axa X vor avea numărul de pași parcurși de model, iar pe axa Y se reprezintă valoarea pierderilor (loss) de-a lungul proceselor de antrenare și evaluare, care măsoară cât de departe sunt predicțiile modelului de valorile reale așteptate.

6.3.1. Modelul de Dialog

Modelul de limbaj antrenat pentru a susține dialog cu utilizatorul a fost supus unui proces de fine-tuning utilizând un set de date relativ mare și divers. Rezultatele acestui proces de fine-tuning sunt vizibile în graficele atașate, care prezintă diverse metrice de evaluare și antrenament ale modelului.

Cele două grafice prezentate în Figura **Eroare! Fără sursă de referință.** ilustrează eroarea modelului atât în timpul antrenamentului (train/loss), cât și în timpul evaluării (eval/loss). În graficul de antrenament, observăm că pierderea începe de la o valoare inițială de aproximativ 2.6 și scade constant pe măsură ce numărul de pași crește, atingând o valoare de aproximativ 1.0. Aceasta sugerează că modelul învață corect, reducând erorile pe setul de antrenament. În contrast, graficul de evaluare arată că pierderea pe setul de evaluare rămâne relativ constantă, pornind de la aproximativ 2.5 și scăzând ușor către sfârșitul antrenamentului. Această constanță indică faptul că

modelul nu suferă de suprainstruire (overfitting) și generalizează bine pe datele noi, deși o scădere mai pronunțată ar fi fost ideală. Comparând cele două grafice, putem concluziona că modelul este bine antrenat și menține o performanță stabilă

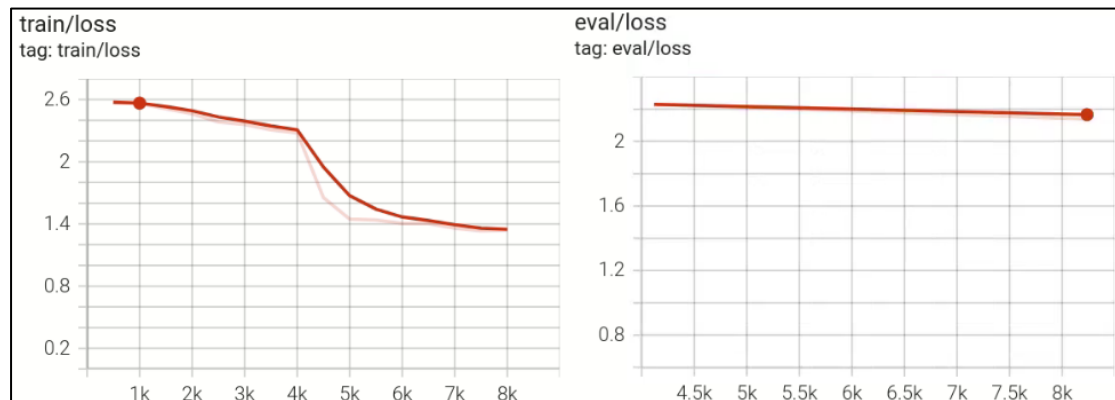


Figura 14: Graficele Evoluției Erorii - Modelul de Dialog

6.3.2. Modelul de Reformulare

Modelul de reformulare a răspunsurilor pe baza emoțiilor a fost antrenat pe un set de date considerabil mai mic decât cel utilizat pentru modelul de dialog. Setul de date pentru acest model a fost generat semi-automat și, din păcate, nu prezintă o varietate adecvată de exemple. Această lipsă de diversitate și volumul redus de date pot influența negativ performanța procesului de fine-tuning, conducând la rezultate sub așteptări.

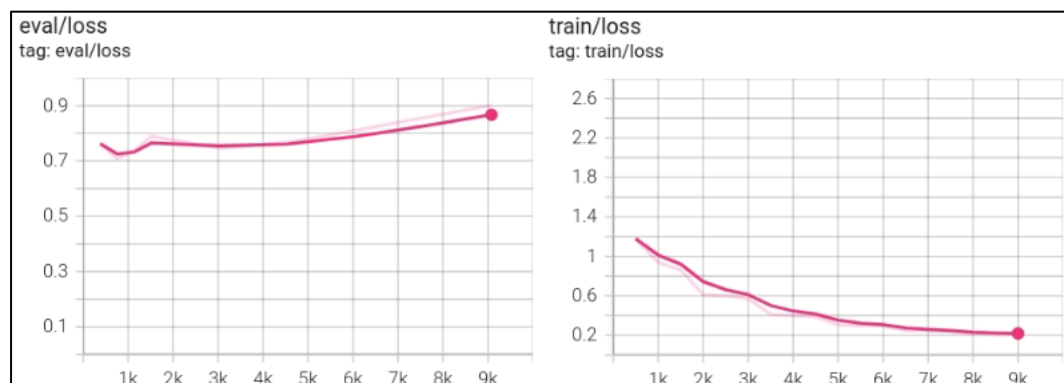


Figura 15: Graficele Evoluției Erorii - Modelul de Dialog

Graficele prezentate, similar cu cele anterioare, ilustrează evoluția metricii de pierdere (loss) atât în timpul antrenamentului (train/loss), cât și în timpul evaluării (eval/loss) pentru modelul de reformulare a răspunsurilor pe baza emoțiilor. Observăm că, pe măsură ce numărul de epoci de antrenament crește, pierderea în timpul antrenamentului scade semnificativ, indicând faptul că modelul învață treptat să ajusteze parametrii pentru a se potrivi mai bine setului de date de antrenament. Aceasta este o tendință așteptată și pozitivă. În contrast, pierderea în timpul evaluării arată o tendință de creștere ușoară după primele câteva epoci, sugerând că modelul ar putea

începe să supraantreneze pe setul de date limitat și lipsit de diversitate. Supra-antrenamentul (overfitting) este o problemă frecventă când setul de date nu este suficient de variat și mare, ceea ce duce la o performanță slabă pe date noi și nevăzute. Aceste grafice subliniază necesitatea unui set de date mai variat și voluminos pentru a îmbunătăți generalizarea modelului și a obține răspunsuri reformulate mai precise și adecvate contextului emoțional al utilizatorului.

6.4. Testarea Experienței Utilizatorului

Testarea experienței utilizatorului (eg. User Experience, UX) reprezintă un proces prin care se evaluează modul în care un utilizator interacționează cu un sistem sau produs, în cazul de față, asistentul personal Odin. Scopul acestei testări este de a înțelege cât de ușor și eficient este pentru utilizatori să folosească asistentul, cât de bine răspunde acesta la comenzi, și cât de natural și captivant este dialogul generat de asistent. Testarea UX implică adesea observarea utilizatorilor reali în timp ce folosesc sistemul, colectarea feedback-ului lor, și analizarea comportamentului și reacțiilor acestora. Pentru a facilita testarea UX, este necesară realizarea unui scenariu bine definit și lipsit de obiectivitate, care va fi prezentat utilizatorilor înainte de începerea testării. Acest scenariu va ghida utilizatorii prin diferitele funcționalități și cazuri de utilizare ale asistentului Odin, asigurându-se că aceștia explorează toate aspectele relevante ale interacțiunii cu sistemul. Pe lângă scenariul de testare, este esențial să identificăm și să definim metricile relevante de evaluat în cadrul interacțiunii cu asistentul. Aceste criterii vor ajuta la obținerea unui feedback structurat și cuantificabil de la utilizatori, iar utilizatorii vor oferi note asistentului pe baza fiecărui criteriu definit.

6.4.1. Scenariul de Test

După cum am menționat anterior, este nevoie de compunerea unui scenariu care să ghideze obiectiv și indirect utilizatorii prin capacitățile asistentului, indicând astfel utilizatorilor ce poate și știe bot-ul să facă, dar nu și ce comenzi trebuie folosite ca să se ajungă la aceste rezultate.

Identificând cazurile de utilizare, avem: prognoza meteo în prezent sau în viitor pentru un anumit oraș, setarea unui reminder pentru o anumită sarcină, gestionarea unei liste de cumpărături, purtarea unui dialog general cu utilizatorul și sistemul de emoții influențate de limbajul și intențiile utilizatorului.

Metricile relevante de evaluare, definite împreună cu profesorul îndrumător, vor fi: înțelegerea mesajelor de către asistent, răspunsurile corecte și la obiect, nivelul de coerență a fluxului de dialog, adaptarea la schimbările de context, calitatea lingvistică a răspunsurilor și nivelul de umanitate al răspunsurilor.

Având acestea în minte, scenariul propus pentru utilizatori este:

„Odin este un asistent personal inteligent care poate automatiza sarcini, având capacitatea să ofere răspunsuri bazate pe emoțiile influențate direct de utilizator prin limbaj și intenții. Odin este capabil:

- *Să salute și să se identifice.*
- *Să poarte conversații generale cu utilizatorul (poate fi întrebat orice).*
- *Să furnizeze detalii despre vreme din prezent sau să găsească prognoza meteo pentru un oraș specificat.*
- *Să seteze un reminder pentru o anumită sarcină.*
- *Să gestioneze o listă de cumpărături (adăugare/citire/ștergere listă).*
- *Să își schimbe starea de spirit în funcție de limbajul utilizatorului și să răspundă pe baza acestor emoții. Utilizatorul poate să vorbească urât, frumos sau să își ceară scuze pentru a afecta emoțiile.*

În urma parcurgerii la îndeplinire a acestor cazuri de utilizare, utilizatorul este rugat să dea o notă asistentului de la 1 la 10 pentru fiecare dintre criteriile:

- *Înțelegerea comenzilor utilizatorului.*
- *Acuratețea răspunsurilor.*
- *Coerența fluxului de dialog.*
- *Adaptabilitatea schimbării contextului în dialog.*
- *Nivelul de umanitate sau naturalețe al conversației.*
- *Satisfacția generală.*

Comunicarea va avea loc în limba engleză, bot-ul va răspunde implicit cu aceleași răspunsuri predefinite, iar pentru activarea modulului care oferă tenta umană răspunsurilor, numit „Humanizer”, trebuie folosită o comandă în acest sens. În cazul interacțiunilor la distanță, se va alege o rețea de socializare unde eu, dezvoltatorul, voi juca rolul unui intermediar, prin redirecționarea comenzilor de la utilizator la bot și răspunsurile de la bot la utilizator. În continuare, dacă nu sunt întrebări, utilizatorul este rugat să aleagă un caz de utilizare din cele de mai sus și să înceapă interacțiunea cu asistentul.”

Sondajul a fost trimis și completat de 4 persoane, care au fost trecute prin fiecare din cazurile de utilizare într-o ordine aleatorie, specificând la finalul dialogului cu asistentul câte o nota obiectiva pentru fiecare dintre criteriile predefinite. Astfel, făcând mediile aritmetice, rezultatele finale la teste indica scorurile:

- *Înțelegerea comenzilor utilizatorului: 6.75/10*
- *Acuratețea răspunsurilor: 7.25/10*

- Coerența fluxului de dialog: 7.5/10
- Adaptabilitatea schimbării contextului în dialog: 8/10
- Nivelul de umanitate sau naturalețe al conversației: 8.25/10
- Satisfacția generală: 8.25/10

Așadar, în urma realizării sondajului de testare, am descoperit câteva slăbiciuni majore ale sistemului. În primul rând, majoritatea mesajelor sau comenzilor date de utilizatori conțin mai multe intenții, iar detectarea a mai mult de o intenție pe mesaj este imposibilă având arhitectura curentă de NLU. Majoritatea utilizatorilor nu sunt și nici nu trebuie să fie familiari cu modul de funcționare sub capota al asistentului, prin urmare majoritatea au fost în sensul exemplului: „*Good morning, Odin! Can you please remind me to buy flowers for my graduating friend next week?*”, în care se observa prezenta atât prezenta intenției *good_morning*, cât și prezenta intenției *set_reminder*. Un alt exemplu întâlnit mai des este cazul în care utilizatorul exprima intenții care influențează mood-ul împreună cu alte intenții ce se referă la task-uri precum „*You are doing a great job Odin, now tell me how the weather is in Cluj tonight.*”.

În al doilea rând, foarte multe mesaje ale utilizatorilor au constat în formulări foarte diferite, greșite sau mult mai complexe decât datele de antrenament, astfel de multe ori asistentul a încurcat intențiile sau a extras entitățile în mod eronat. Spre exemplu intenția *set_reminder* a fost antrenată pe exemple de genul: „*Remind me to ... in ... hours/minutes*”; dar un utilizator a formulat mesajul gen: „*Create a reminder named "gym" for tomorrow at 10 PM*”. Au fost multe instanțe de formulări ale mesaje care nu au fost luate în considerare în dezvoltarea aplicației și a algoritmilor, prin urmare au fost multe rateuri din partea asistentului.

În ciuda acestor rezultate negative, majoritatea utilizatorilor care au făcut parte din test au concluzionat că asistentul este distractiv de folosit, deoarece nu are limitele regăsite în majoritatea soluțiilor similare, fapt ce se reflecta în notele ridicate asigurate criteriilor de satisfacție și naturalețe. Utilizatorii au apreciat cel mai mult natura umană a răspunsurilor date de tentele unice de sarcasm sau pozitivitate oferite de componenta de umanizare, care au provocat amuzament, iar dialogul imprecis dar în același timp imprevizibil a ținut unii utilizatori prinși în conversație cu scopul de a vedea ce lucru amuzant și în afara contextului va spune asistentul în continuare.

Capitolul 7. Manual de Instalare si Utilizare

Instalarea și configurarea unui sistem distribuit, precum asistentul Odin, reprezintă un proces complex care poate necesita două stații de lucru sau modificarea configurării pentru a funcționa pe una singură. Deși resursele hardware necesare pentru un bot care înțelege limbajul natural, automatizează sarcini și răspunde într-un mod natural sunt mai mici comparativ cu asistenții care rulează pe cloud, ele sunt totuși semnificative pentru un consumator obișnuit. Pe lângă acestea, configurarea software implică instalarea unui număr mare de librării și instrumente auxiliare pentru a asigura funcționarea corectă și completă a bot-ului. În continuare, vor fi detaliate cerințele hardware și software, precum și configurările necesare pentru variabilele bot-ului.

7.1. Resursele Hardware

Pentru rularea acestui sistem sunt necesare resurse performante în toate componentele de calcul ale unui PC: CPU, RAM și VRAM. Suportul nucleului necesită un procesor performant și o cantitate decentă de memorie RAM, esențiale pentru sarcinile principale de înțelegere a limbajului natural (NLU) și automatizare a task-urilor. În cazul de față, nucleul asistentului rulează pe un laptop Windows cu următoarele specificații: *CPU Intel(R) Core(TM) i7-5600U CPU @ 2.60GHz* și *16 GB RAM DDR4*. Deși timpul de procesare a comenzilor cu astfel de resurse este mic, performanța ar putea fi îmbunătățită, având în vedere că există o latență introdusă de celelalte componente.

Componenta de umanizare, care, deși nu este absolut necesară, este importantă pentru a furniza o experiență plăcută și autentică cu asistentul, rulează pe un PC Ubuntu cu specificațiile: *CPU AMD® Ryzen 5 2600 cu 6 Core, 32 GB RAM DDR4, 1 x NVIDIA GeForce RTX 2060, 2 x NVIDIA GeForce GTX 1660 SUPER și 1 x NVIDIA GeForce GTX 1070*.

Pentru a asigura securitatea comunicării dintre aceste două componente, se recomandă configurarea unui server *OpenVPN*, fie utilizând un hardware specializat, fie un router care suportă acest serviciu.

7.2. Resursele Software

În acest subcapitol vom detalia instalarea și configurarea uneltelor software necesare pentru rularea locală a asistentului, atât pentru nucleu, cât și pentru serverul opțional de umanizare de la distanță.

7.2.1. Nucleul

Pentru a instala și configura corect asistentul personal Odin, începem cu descărcarea și instalarea Python 3.10 de pe site-ul oficial. După instalarea Python, creăm și activăm un mediu virtual pentru a evita conflictele și erorile între dependențe.

Acest lucru se face cu comenzile „*python -m venv odin_env*” urmată de „*.\odin_env\Scripts\activate*” pe Windows. Următorul pas este instalarea dependențelor specificate în fișierul *requirements.txt* cu comanda „*pip install -r requirements.txt*”. După aceasta, instalăm Rasa cu comenzile „*git clone https://github.com/RasaHQ/rasa.git*”, „*cd rasa*” și „*pip install -e .*”. În continuare, configurăm IP-urile pentru apelurile la serverul de umanizare; dacă rulează pe aceeași mașină cu nucleul, este suficient să folosim *localhost*, dacă nu, trebuie înlocuit cu IP-ul din rețeaua locală a mașinii de umanizare. Mai departe, configurăm un client OpenVPN, încărcând fișierul de configurare extras din serverul OpenVPN care rulează pe router. Acest lucru permite funcționarea IP-urilor locale pentru apelarea serverului, chiar dacă nucleul și serverul nu sunt în aceeași rețea locală.

7.2.2. VPN-ul

Pentru a asigura o cale securizată și fiabilă de comunicare între nucleu și server, este necesară configurarea serverului OpenVPN din setările routerului. După configurarea serverului OpenVPN, salvați fișierul generat, care conține parolele și cheile de criptare necesare. Acest fișier va fi încărcat ulterior în clientul OpenVPN de pe dispozitivul care rulează nucleul. După configurarea VPN-ului, putem testa comunicarea folosind terminalul și comanda ping de la dispozitivul care rulează nucleul către serverul de umanizare. Este recomandat să ieșiți din rețeaua locală a serverului și să repetați testul ping pentru a vă asigura că serviciul VPN funcționează corect și recunoaște conexiunile externe. Acest test confirmă că toate comunicațiile dintre nucleu și serverul de umanizare sunt securizate și funcționează conform așteptărilor.

7.2.3. Serverul de Umanizare

Pentru configurarea serviciului de umanizare, care rulează două modele de limbaj și unul de TTS, este necesară instalarea și configurarea uneltelor *CUDA*, care permit interacțiunea între modele și plăcile grafice. *CUDA* (Compute Unified Device Architecture) este o platformă de calcul paralel și un model de programare creat de *NVIDIA*, care permite utilizarea procesorului grafic (*GPU*) pentru a accelera calculele intense. Instalarea *CUDA* pe *Ubuntu* poate fi realizată prin actualizarea sistemului cu comanda „*sudo apt-get update*” urmată de „*sudo apt-get upgrade*”. După aceasta, se descarcă și se instalează pachetul *CUDA* utilizând comanda „*wget*” pentru a descărca pachetul de pe site-ul oficial *NVIDIA*, urmată de „*sudo dpkg -i*” pentru instalarea pachetului. După instalare, trebuie să adăugăm cheia publică *NVIDIA* folosind „*sudo apt-key adv --fetch-keys*” și să actualizăm din nou sistemul cu „*sudo apt-get update*”, apoi instalăm efectiv *CUDA* cu „*sudo apt-get install cuda*”.

Odată instalat *CUDA*, trebuie să adăugăm variabilele de mediu necesare pentru a putea folosi *CUDA*. Acest lucru se face adăugând liniile

„*export PATH=/usr/local/cuda-10.2/bin:\$PATH*” și

„*export LD_LIBRARY_PATH=/usr/local/cuda10.2/lib64:\$LD_LIBRARY_PATH*” în fișierul *.bashrc* al utilizatorului și rulând „*source ~/.bashrc*” pentru a reîncărca fișierul de configurare.

Similar cu pașii efectuați la configurarea nucleului, este necesară instalarea Python 3.10. Instalarea Python 3.10 pe Ubuntu implică adăugarea repository-ului pentru Python 3.10 cu „*sudo add-apt-repository ppa:deadsnakes/ppa*” și actualizarea listei de pachete cu „*sudo apt-get update*”. Apoi, se instalează Python 3.10 folosind comanda „*sudo apt-get install python3.10*” împreună cu pachetele necesare pentru crearea și gestionarea mediilor virtuale, *python3.10-venv* și *python3.10-dev*.

Pentru a configura un mediu virtual și a instala dependențele, se creează un mediu virtual cu „*python3.10 -m venv myenv*” și se activează cu „*source myenv/bin/activate*”. După activarea mediului virtual, se instalează toate dependențele specificate în fișierul *requirements.txt* utilizând comanda „*pip install -r requirements.txt*”.

7.3. Rulare și Utilizare

Pentru a rula toate componentele distribuite, sunt necesare câteva comenzi, dar au fost create și scripturi ajutătoare pentru a simplifica procesul.

În primul rând, nucleul poate fi pornit din terminal din directorul rădăcină al asistentului cu comanda „*python3 run.py*”, care instanțiază și rulează subprocese pentru serviciul de NLU, cel de acțiuni și cel de reamintiri, și redirecționează ieșirea standard a tuturor acestor procese într-o singură consolă. Rularea controlorului de chat se va face separat, cu comanda *python3 chat.py*, deoarece acesta necesită un terminal separat pentru claritatea comunicării dintre bot și utilizator.

În al doilea rând, pentru rularea serviciilor de umanizare, se folosesc trei comenzi simple: „*python3 dialogue_server.py*”, „*python3 rephrasing_server.py*” și „*python3 tts_server.py*”. Aceste comenzi vor porni serverele necesare pentru procesarea dialogului, reformularea răspunsurilor și conversia text-voce, respectiv.

Indicațiile pentru utilizarea asistentului au fost detaliate suficient în scenariul realizat pentru testarea experienței utilizatorului găsit în capitolul 6.4.1, unde se prezintă în amănunt cazurile de utilizare și instrucțiunile de folosire ale asistentului Odin.

Capitolul 8. Concluzii

În concluzie, în urma procesului îndelungat și dificil de dezvoltare a asistentului Odin, au fost atinse anumite obiective principale și secundare, dar încă ar mai fi mult de lucru pentru obținerea unei soluții stabile și fiabile, utilizabilă în viața de zi cu zi.

Odin a reușește să automatizeze cu succes sarcinile zilnice și să ofere o capacitate de dialog uman captivant și distractiv. Mai mult, capacitatea sa de a purta conversații umane și de a răspunde în funcție de emoțiile exprimate de utilizator îl diferențiază semnificativ de alte soluții existente pe piață. Această abordare bazată pe emoții îi conferă un caracter mai uman și mai natural, eliminând limitările rigide ale răspunsurilor predefinite și aducând o experiență mai autentică utilizatorilor.

Cu toate acestea, există și puncte în care Odin a eșuat. Una dintre principalele probleme este capacitatea sa de a generaliza datele noi și de a identifica diferitele formulări ale comenzilor pe care utilizatorii noi le-ar putea avea. Odin nu reușește întotdeauna să înțeleagă corect diversele modalități în care utilizatorii își exprimă intențiile, ceea ce poate duce la răspunsuri inadecvate sau la eșecuri în executarea sarcinilor. De asemenea, Odin poate detecta o singură intenție per mesaj, ceea ce reprezintă o limitare semnificativă, deoarece utilizatorii tind să formuleze mesaje mai complexe care pot conține multiple intenții. Un alt aspect dezamăgitor este modul frecvent de răspuns al bot-ului, care poate părea pueril și lipsit de atenție la contextul conversației. Deși unele răspunsuri sunt distractive, Odin comunică uneori ca un copil de 10 ani, nereușind să mențină un nivel adecvat de maturitate, atenție și coerență în dialog.

În ciuda faptului că multe unelte folosite în dezvoltarea asistentului Odin vin gata de utilizare, cum ar fi pipeline-ul NLU și restul abilităților oferite de framework-ul Rasa, sau bibliotecile de simplificare a procesului de fine-tuning oferite de librăria Transformers de la Hugging Face, au avut loc multe contribuții din partea mea, extinzând astfel funcționalitățile disponibile de la bun început cu mai multe contribuții cheie. Printre principalele mele contribuții se numără:

- Dezvoltarea algoritmilor ajutători de conversie a timpului natural pentru găsirea prognozei meteo.
- Implementarea modului de reformulare prin generarea unui set de date unic și reglarea fină a modelului pe baza acestuia
- Dezvoltarea unui sistem de reamintiri care trimite mesajele asincron în funcție de programarea în viitor a acestora de către utilizator
- Integrarea tuturor componentelor într-un mediu distribuit și asigurarea comunicării sigure fiabile între acestea prin VPN

Ca și dezvoltări ulterioare, pentru a aduce asistentul personal Odin la nivelul următor, este indicată folosirea unui pipeline mai avansat de NLU, care ar putea înțelege mesaje mai complexe cu multiple intenții. Aceasta ar permite asistentului să gestioneze mai bine situațiile în care utilizatorii formulează cereri complexe și să ofere răspunsuri mai precise și relevante. De asemenea, seturile de date pentru modelele de limbaj pot fi revizuite și îmbunătățite. Un set de date mai variat și mai bine etichetat ar contribui semnificativ la creșterea performanței modelelor după procesul de fine-tuning. În ultimul rând, nu ar strica implementarea unui modul de conversie speech-to-text. Aceasta ar elimina necesitatea utilizatorului de a interacționa cu un dispozitiv printr-o tastatură, facilitând o comunicare strict vocală între utilizator și Odin.

Bibliografie

- [1] D. K. V. Kulhalli, „Personal Assistant with Voice Recognition Intelligence (PARI),” in *IJERTV10N1SPL*, 2021.
- [2] N. Zierau, „The Smart Personal Assistant: An Overview,” 2021.
- [3] „A Brief Overview on AI and Machine Learning in SPAs,” 2021.
- [4] U. R. Peng Cheng, „Personal Voice Assistant Security and Privacy - A Survey,” *Proceedings of the IEEE*, 2022.
- [5] W. X. Zhao, „A Survey of Large Language Models,” *arXiv*, 2023.
- [6] Y. S. H. J. Yu Wang, „A Bi-model based RNN Semantic Frame Parsing Model for Intent Detection and Slot Filling,” *arXiv*, 2018.
- [7] D. V. V. A. N. Tanja Bunk, „DIET: Lightweight Language Understanding for Dialogue Systems,” *arXiv*, 2020.
- [8] C. H. J. H. X. T. T. Q. Z. Z. T.-Y. L. Yi Ren, „NaturalSpeech: End-to-End Text to Speech Synthesis with Human-Level Quality,” *arXiv*, 2022.
- [9] K. D. E. G. G. I. G. L. H. A. A. J. M. R. M. S. O. J. W. Edresson Casanova, „XTTS: a Massively Multilingual Zero-Shot Text-to-Speech Model,” *arXiv*, 2024.
- [10] J. F. N. P. A. N. Tom Bocklisch, „Rasa: Open Source Language Understanding and Dialogue Management,” *arXiv*, 2017.
- [11] S. Blog, „Fine-tuning Large Language Models,” Available: <https://www.superannotate.com/blog/llm-fine-tuning>.
- [12] A. S. N. P. N. U. J. J. L. G. A. N. K. Ł. & P. Vaswani, „Attention Is All You Need,” *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017.

